bigT

P2P settlement — CNY rails demo (WeChat / Alipay / Bank)

P2P settlement · WeChat / Alipay / bank
Step-by-step demo guide — the CNY rails
captured 2026-10-12T10:45:51.946Z · wechat `swap-a13efab115858720` · alipay `swap-f8226be10089ddad` · bank `swap-ad4639ba7976dd97`

Status: **built and captured**. Every screenshot below was taken from the real wallet UI driving the real `p2p-engine` — one engine, no mocks. The three CNY legs were captured by `e2e/capture-p2p-cny.mjs` (a fresh wallet pair per rail). The rail is a data field, not a fork (`docs/p2p.md` §7): the same 2-of-3 escrow, the same PQ-signed state machine, three different fiat legs.

The seller's receipt can also be **read automatically from their own phone** through the on-phone agent — the screen tree for Alipay/bank, or WeChat's own "收款到账" push for WeChat (which hides its UI from accessibility). That is §8 Device-side verification.

Companion documents: docs/p2p.md (design, protocol & threat model), engine source `services/p2p-engine/src/`.

---

0. What this demo proves (read first)

1. **One state machine, three fiat rails.** `wantRail ∈ {wechat, alipay, bank}` decides the whole fiat leg; all three use `instructions` / `proof` / `confirm` / `release` / `complete` (`state.ts:9-28`). Everything before `MATCHED` and the escrow lock is identical.
2. **2-of-3 P2SH escrow on Bigtangle L0.** Seller, buyer and engine hold one key each; any two satisfy the script. Nobody — including the engine — can move the tokens alone. The engine proves the lock on chain and fails closed if the transaction is not `CONFIRMED` at the derived escrow address.
3. **The seller's own confirmation is the fiat signal.** There is no webhook for a WeChat/Alipay/bank transfer. `PAYMENT_CLAIMED` is only the buyer's claim (流水号 + optional receipt); tokens stay locked until the seller signs `confirm` after checking their own account — the UI repeats this as a red warning above the *Confirm received* button.
4. **PII never reaches the order book or the chain.** `GET /public/orders` carries only the trade terms. The seller's 收款 account (WeChat ID, Alipay account, or bank card + bank name) is revealed party-scoped by `POST /swaps/:id/payment-instructions`; only a per-swap `remark` code plus (optionally) a receipt `sha256` are stored on the swap event.
5. **Every mutating call is signed with the party's PQ (ML-DSA-87) `did:key`** — the same wallet key that signs chain transactions — over the sha256 of the canonical (sorted top-level keys) JSON body, plus a nonce + timestamp (±5 min): replay-protected, rate-limited to 10 signed requests per DID per

minute (`sign.ts:17-19`). The demo's final legs (`release` / `complete`) are signed by the engine signer (`SETTLEMENT_ENGINE_DID` , an ML-DSA did:key ≈3.5 kB); nothing fires them automatically yet, so the capture script stands in for that operator job (§6.1).

6. **The receipt can be machine-read from the payee's own phone (§8).** The seller pairs the on-phone agent once; the engine then reads the seller's own WeChat/Alipay/statement via `POST /swaps/:id/device-read` — the screen tree, or, for WeChat (which hides its UI from accessibility), WeChat's own "收款到账" push — and, under `SETTLEMENT_CNY_AUTO_RELEASE_ON=seller_receipt` , machine-confirms without the manual tap.

---

1. State machine — who advances each step

CNY rails (`wechat` / `alipay` / `bank` — seller-confirmed):

#	Status	Action (HTTP)	Who must sign	CNY-specific evidence
1	`MATCHED`	`POST /orders/:id/match`	buyer	`receiveAddress` (token payout address)
2	`ESCROW_LOCKED`	`POST /swaps/:id/transitions action=escrow_lock`	**seller**	`txHash` (escrow lock proof)
3	`PAYMENT_PENDING`	`POST /swaps/:id/payment-instructions`	buyer	`remark` (12 hex chars, per-swap)
4	`PAYMENT_CLAIMED`	`POST /swaps/:id/proof`	buyer	`txId` 流水号, optional `receipt` + `receiptSha256`
5	`PAYMENT_VERIFIED`	`POST /swaps/:id/confirm` · or `POST /swaps/:id/device-read` + `POST /swaps/:id/verify` under policy (§8)	**seller**	the seller's own statement check
6	`ESCROW_RELEASED`	`POST /swaps/:id/transitions action=release`	**engine**	`txHash` (release proof)
7	`COMPLETED`	`POST /swaps/:id/transitions action=complete`	**engine**	— (seller already holds the CNY)

Freeze paths: either party may `POST /swaps/:id/dispute` while `PAYMENT_PENDING` / `PAYMENT_CLAIMED` ; the admin then resolves with `outcome=release|refund` . `expire` / `refund` / `cancel` stay available before release so a failed deal can always return the tokens.

---

2. The rails and how they are configured

Rail (`wantRail`)	Buyer pays	Seller verifies	Tokens released by
`wechat`	WeChat Pay to seller's WeChat ID	seller's own 流水 + remark	engine `release` + `complete`
`alipay`	Alipay to seller's account	seller's own 流水 + remark	engine `release` + `complete`
`bank`	bank transfer + remark	bank statement + remark	engine `release` + `complete`

Engine environment (`services/p2p-engine/src/server.ts`):

```
SETTLEMENT_CNY_RAILS=wechat,alipay,bank  # unset → all three; "" → rail off
SETTLEMENT_CNY_REMARK_TTL=900            # seconds → payBy deadline in instructions
SETTLEMENT_ENGINE_DID=did:key:z446Gmp1… # ML-DSA-87 did:key (≈3.5 kB); only signer of release/complete
SETTLEMENT_ENGINE_PUBKEY=050101010a20…  # prefixed ML-DSA pub (hex) → 2-of-3 escrow address derivation
SETTLEMENT_STORE=mem|pg                  # mem for demos, pg for production
SETTLEMENT_ADMIN_TOKEN=adm               # /orders, /swaps, dispute/resolve
CORS_ORIGIN=http://localhost:18081       # wallet web app origin
```

3. Running the demo yourself

```
# 1. workspace packages the engine imports
for pkg in chain-discovery did p2p-protocol record-sig; do ( cd packages/$pkg && npm run build ); done

# 2. engine bundle (esbuild → dist/server.bundle.mjs)
( cd services/p2p-engine && npm run build && npm run bundle )

# 3. wallet web app with the engine URL inlined by `expo export`
( cd expo-app && EXPO_PUBLIC_P2P_ENGINE_URL=http://localhost:18089 npm run web:build )
echo http://localhost:18089 > e2e/web-build/.p2p-url

# 4. CNY legs: 14 screenshots per rail → assets/screenshots/p2p-cny-{wechat,alipay,bank}-NN-*-en.png
( cd e2e && RAILS=wechat,alipay,bank node capture-p2p-cny.mjs )

# 5. render the PDF
node docs/p2p-demo/scripts/gen-p2p-all-pdf.mjs
```

Or all of it through the one entry point: `./p2pdemo.sh capture walkthrough`

(see `p2pdemo.sh --help`). The captures are self-contained: no L0/L1 — a CNY

swap has no chain-side dependency beyond the escrow lock proof the engine skips

when `deps.chain` is unset. `RAILS=wechat` (or `alipay` / `bank`) limits the run

to one rail.

Run facts used in the screenshots below:

```
{ "wechat": { "swapId": "swap-a13efab115858720", "txId": "42034516068EAEC2", "remark": "f71ade2d23e9"
},
  "alipay": { "swapId": "swap-f8226be10089ddad", "txId": "200E31282969161F6B32A2CF1C93", "remark":
"91b6eeced27f" },
  "bank":   { "swapId": "swap-ad4639ba7976dd97", "txId": "30291CF907571417", "remark": "20e892c372ce"
},
  "capturedAt": "2026-10-12T10:45:52Z" }
```

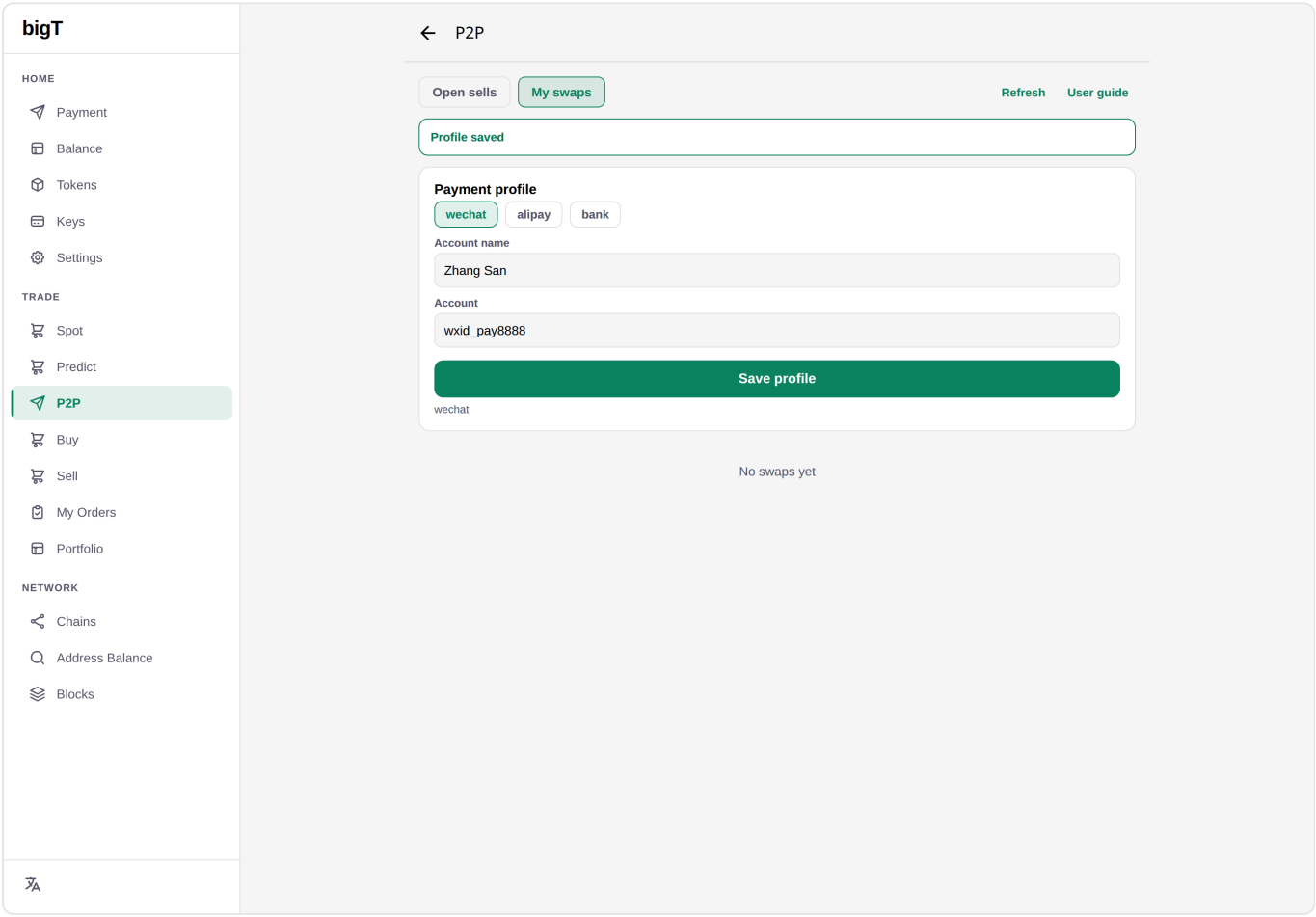

---

4. Walkthrough — WeChat Pay (CNY)

The CNY rails need one extra step at the start (the seller's collection profile) and replace the webhook leg with `instructions → proof → confirm`. The seller is paid **first** (the buyer transfers CNY directly to the seller's own WeChat account); the tokens are released only after the seller confirmed.

Step 1 — Seller saves a collection profile

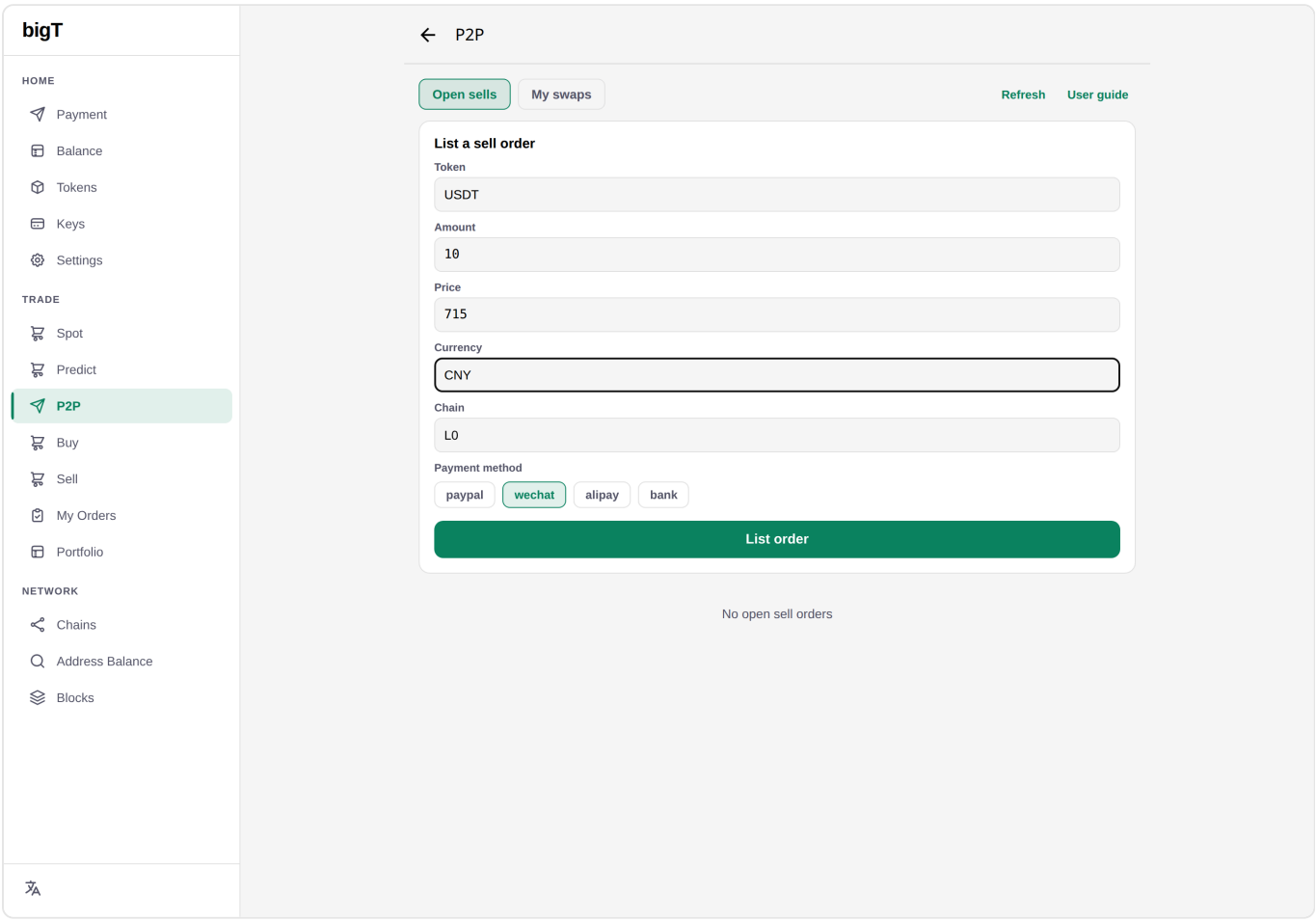
The seller opens **P2P** → **My swaps**, picks the rail (`wechat`), and enters the 收款 name and account (WeChat ID). The profile is stored server-side under the seller's DID and is **not** part of the public order book — it is only revealed to the matched buyer by the instructions call.



Step 2 — Seller lists 10 USDT for 715 CNY on the wechat rail

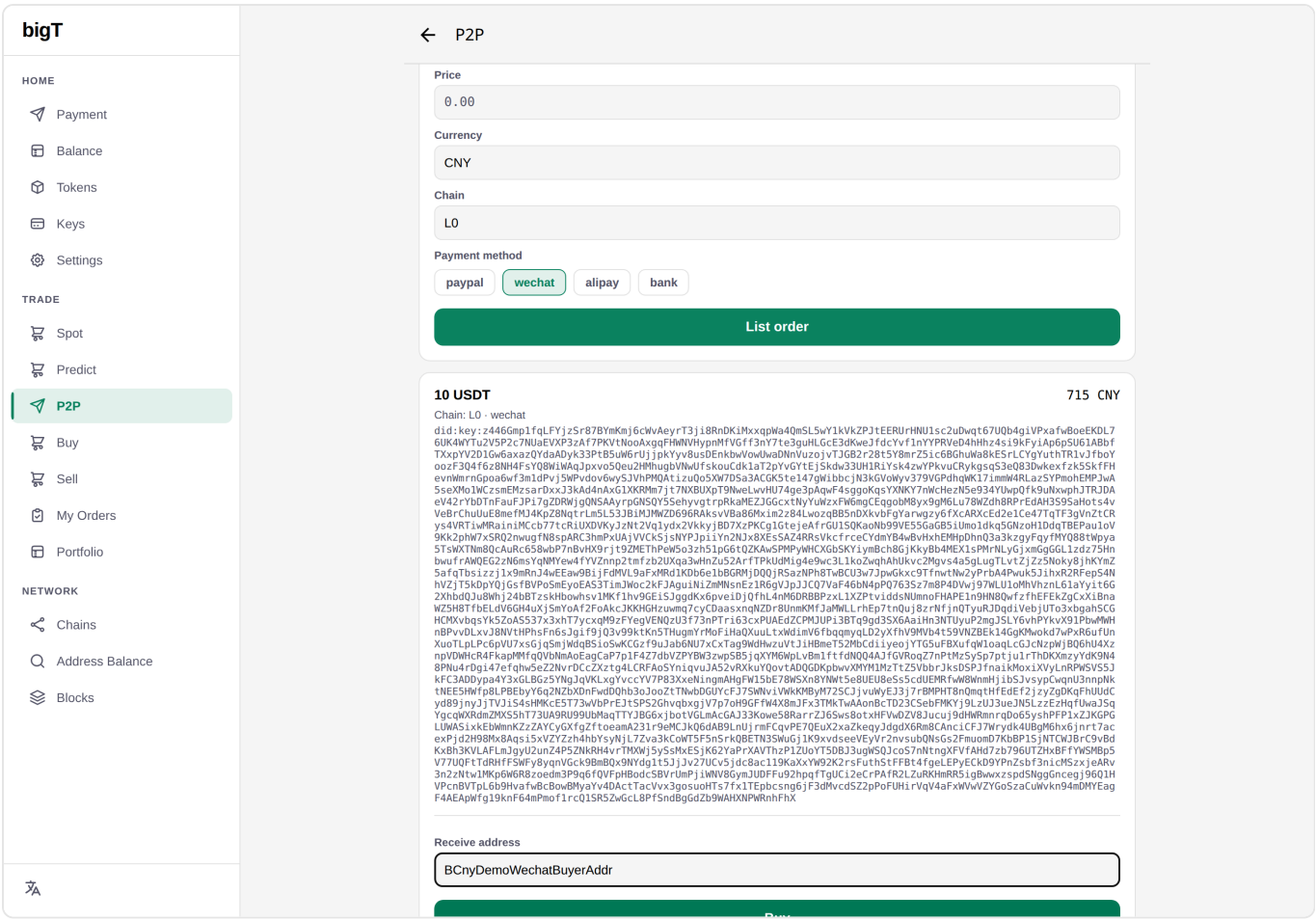
Open sells → **List a sell order**: token `USDT`, amount `10`, price `715`, currency `CNY`, chain `L0`, payment method **wechat** (selecting a CNY rail switches the currency to CNY). The signed `POST /orders` payload carries no PII:

```
{ "sellerDid": "did:key:z446G…(ML-DSA did, ≈3.5 kB)", "giveToken": "USDT",
  "giveAmount": "10", "giveChain": "L0", "wantCurrency": "CNY",
  "wantAmount": "715", "wantRail": "wechat", "validUntil": 1791510000,
  "nonce": "…16+ chars…", "timestamp": 1791509100000,
  "signature": "…hex SignatureBundle over sha256(canonical JSON)…"}
```

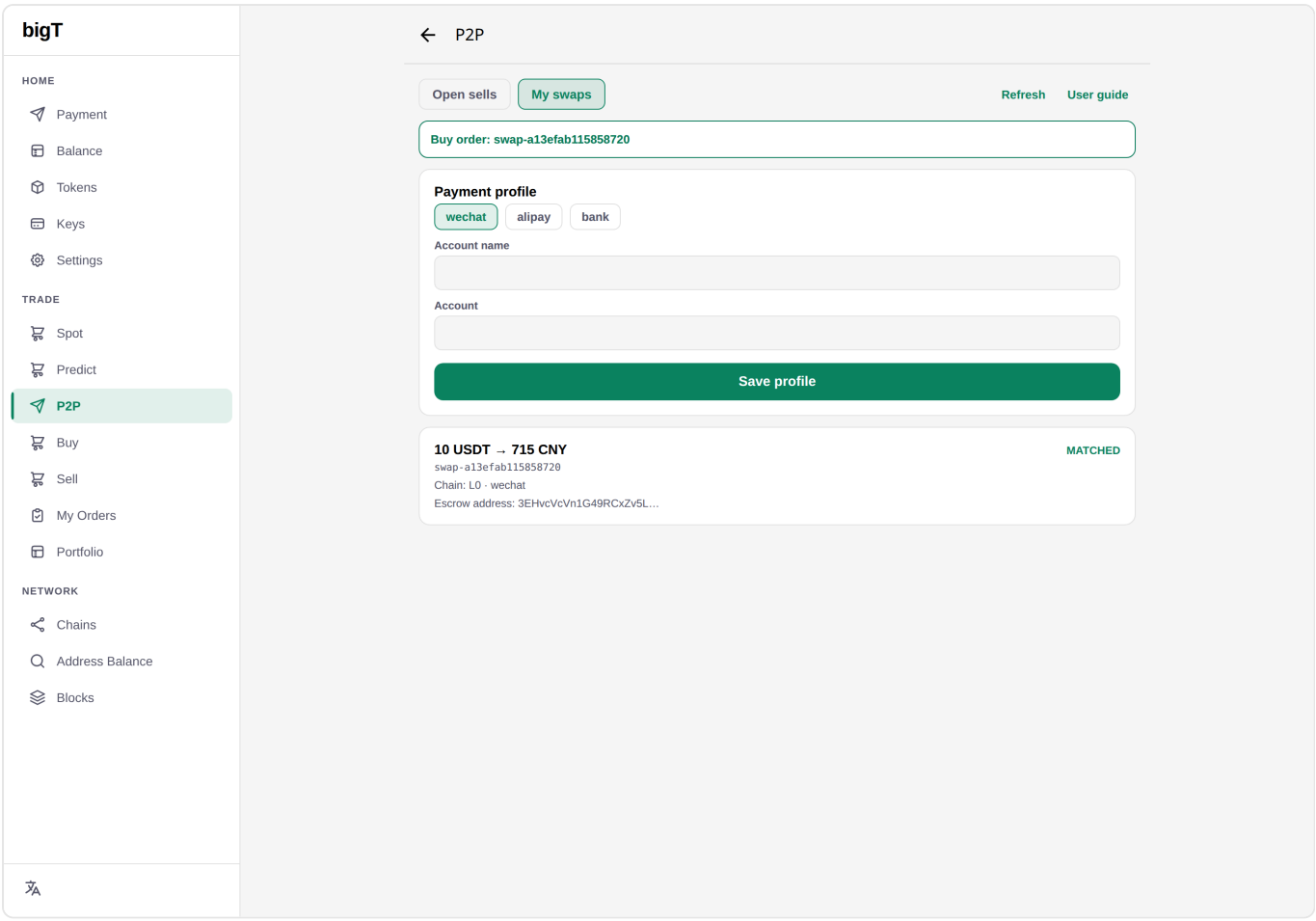


Step 3 — Buyer matches the order

The buyer (fresh wallet, no profile needed) sees the order in **Open sells** and opens the buy panel. On a CNY order only a receive address is required:

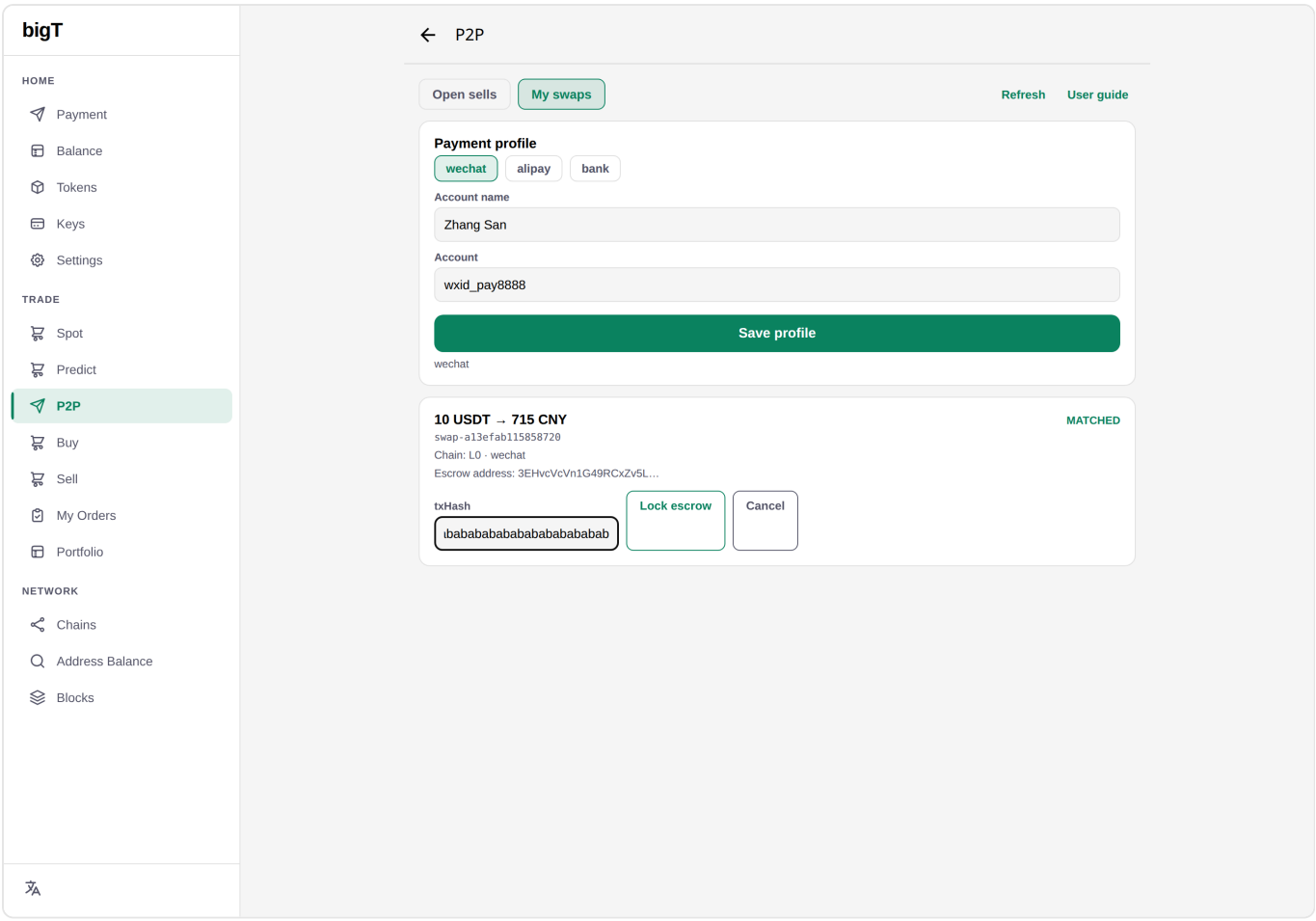


After **Confirm buy** the engine mints the swap at `MATCHED` and derives the 2-of-3 escrow address from the seller's and buyer's PQ dids plus the engine's prefixed ML-DSA public key (`SETTLEMENT_ENGINE_PUBKEY`):

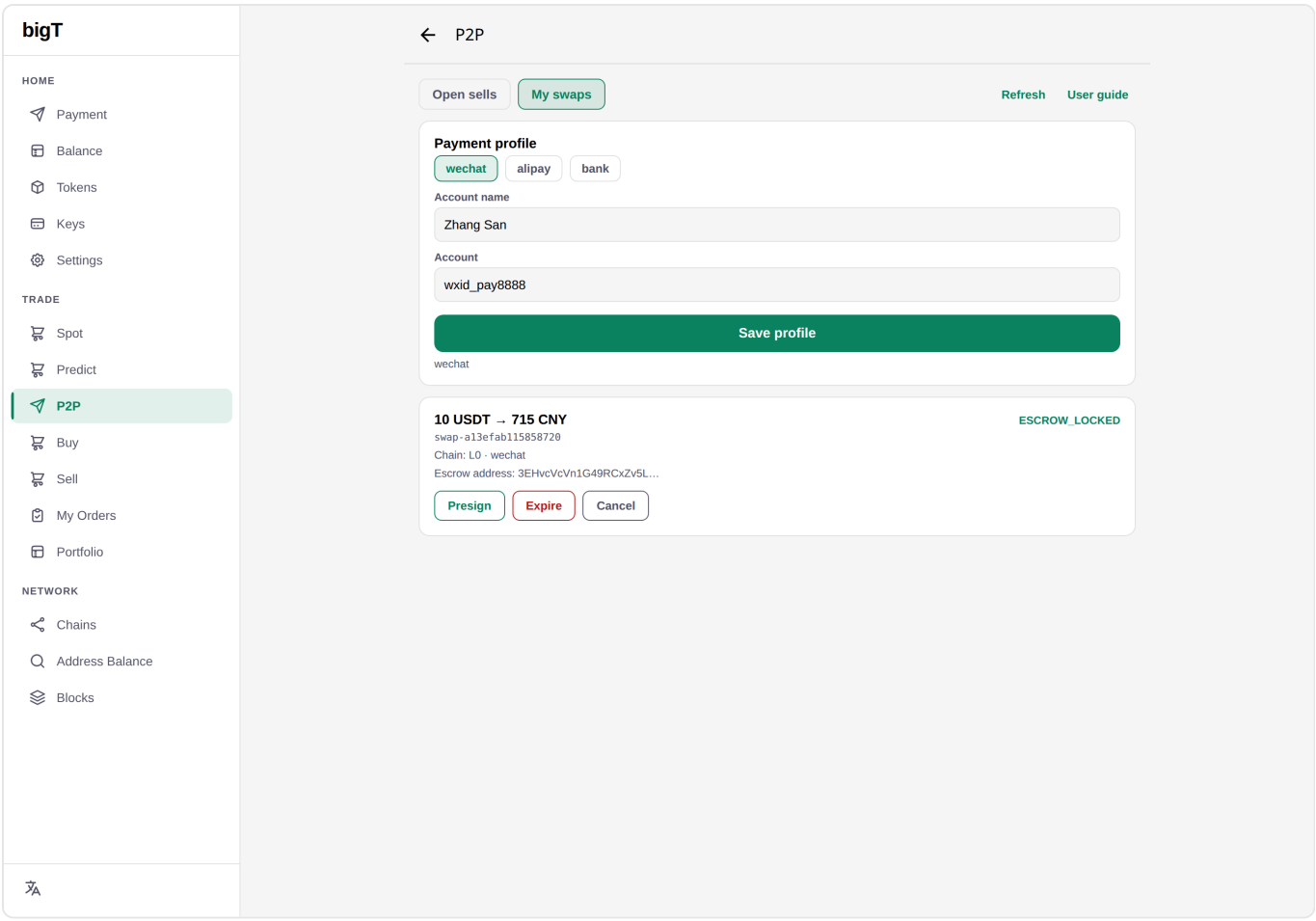


Step 4 — Seller locks the token escrow

Back on the seller's **My swaps** tab, the seller pastes the escrow funding transaction hash and presses **Lock escrow** — a seller-signed `action=escrow_lock` transition. With `deps.chain` configured the engine verifies the lock on chain; in the demo (`SETTLEMENT_STORE=mem`, no chain) it is recorded as-is.



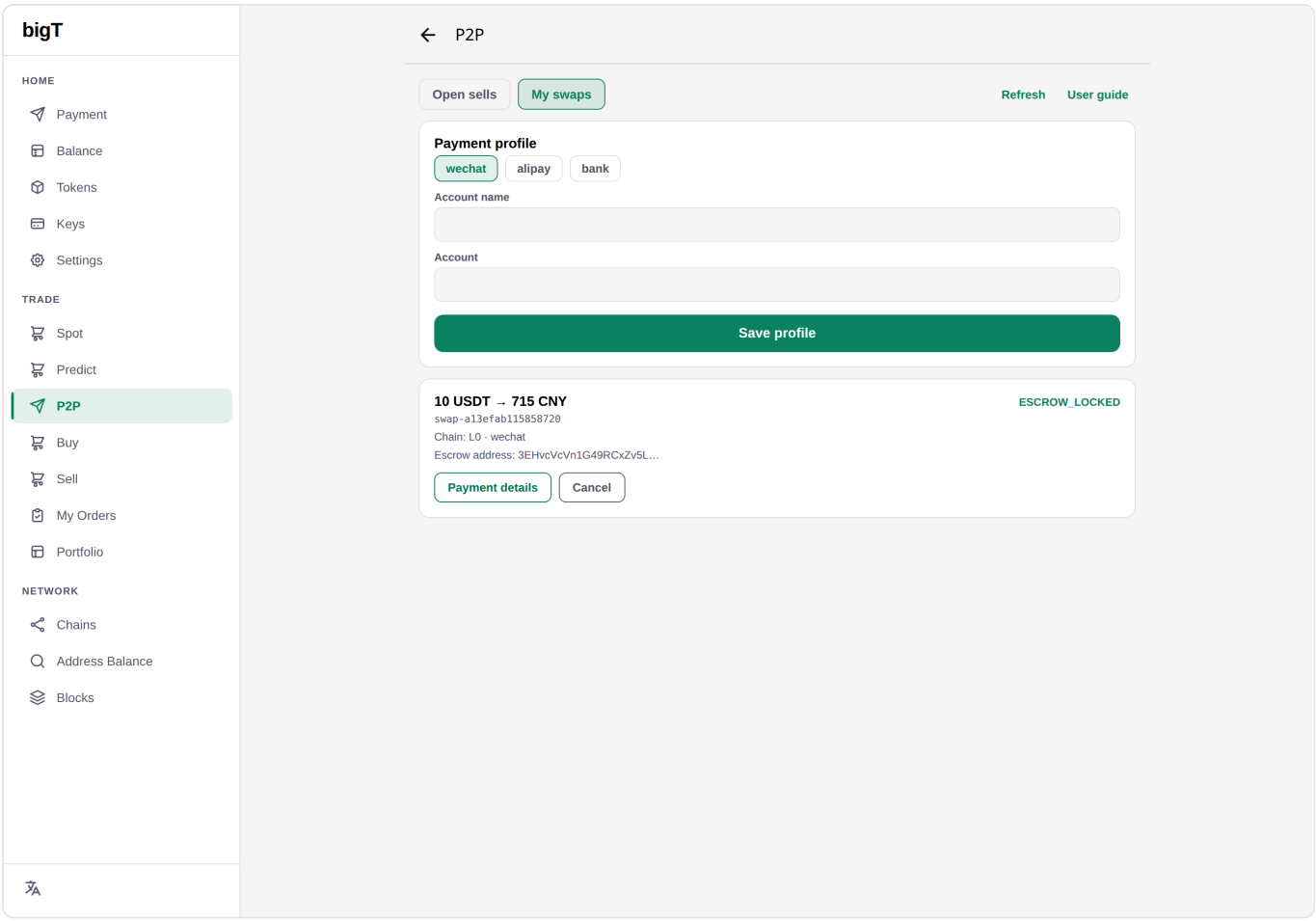
Status advances to `ESCROW_LOCKED`. **Nothing has been paid yet** — this is the point of no return for the seller's tokens.



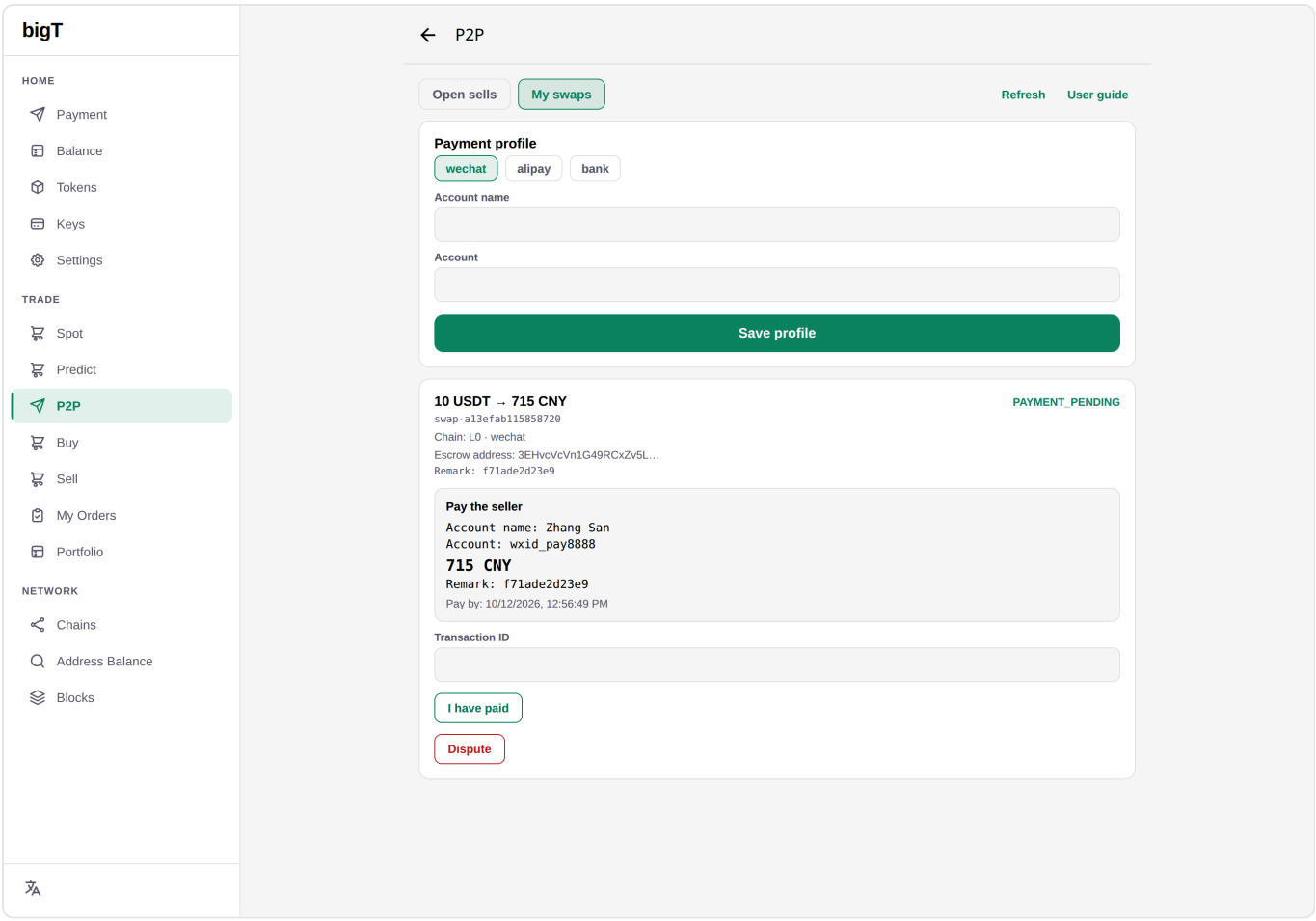
Step 5 — Buyer pulls the signed payment instructions

The buyer's card now shows a **Payment details** button. Pressing it issues the buyer-signed `POST /swaps/:id/payment-instructions`, which is idempotent while `PAYMENT_PENDING` and reveals the seller's profile + a fresh per-swap `remark` the transfer *must* carry:

```
{ "method": "wechat", "rail": "wechat", "accountName": "Zhang San",
  "account": "wxid_pay8888", "amount": "715", "currency": "CNY",
  "remark": "f71ade2d23e9", "payBy": 1791510060 }
```

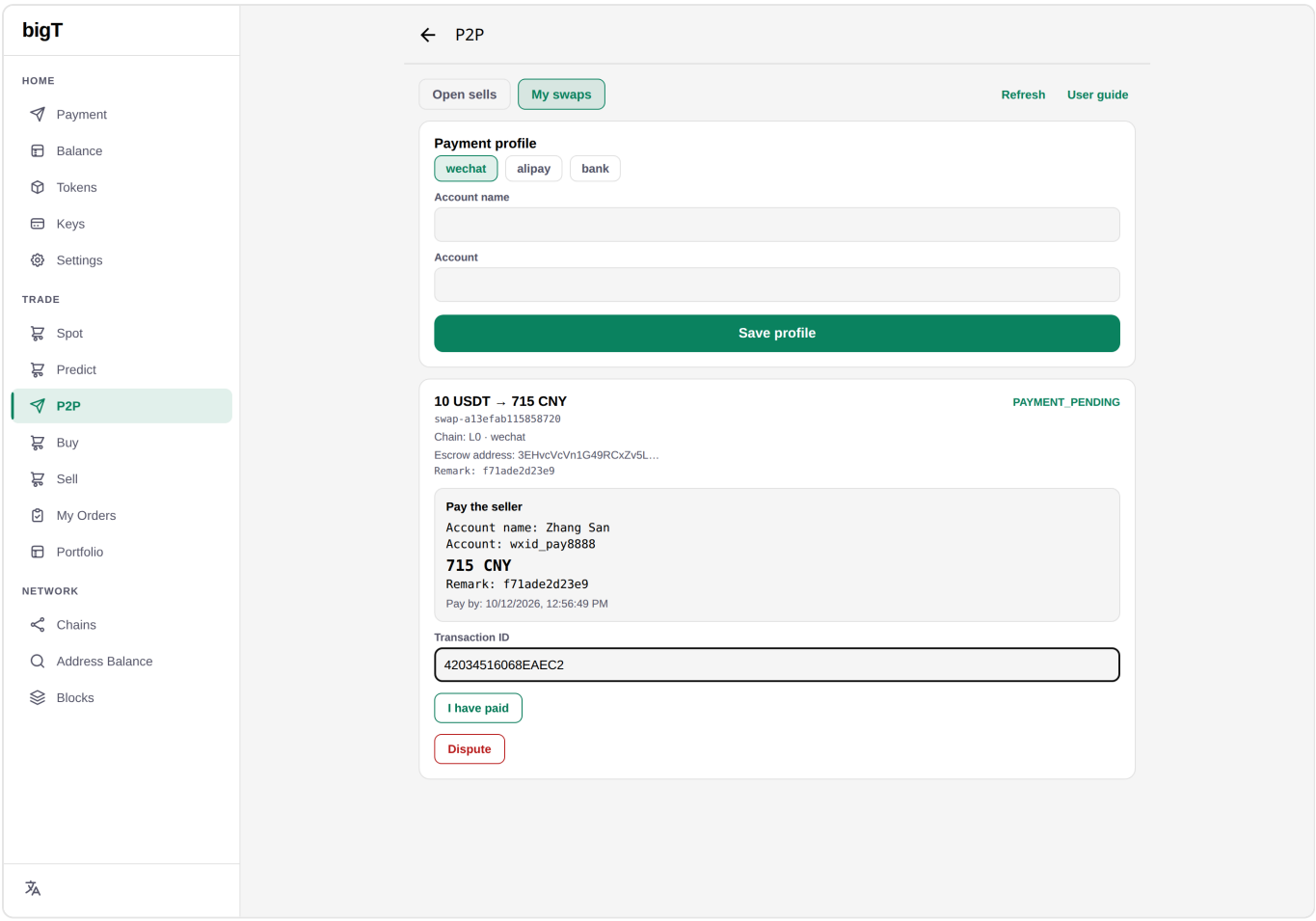


The instructions render **inline in the swap card** (amount, account, remark, deadline) — plus an optional QR payload when the profile stores one:

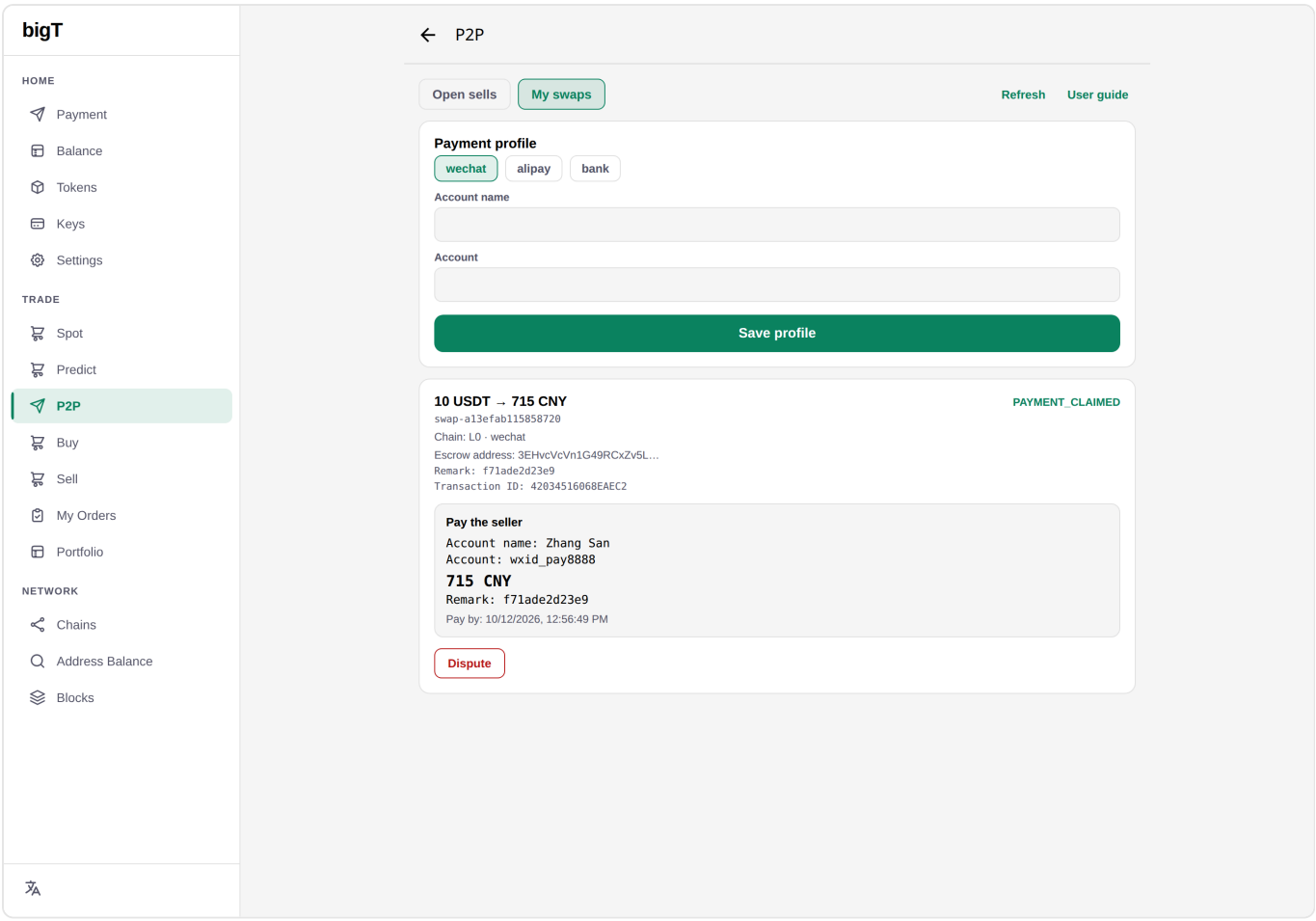


Step 6 — Buyer pays in WeChat and claims with the 流水号

The buyer transfers `715 CNY` in their own WeChat app, pastes the transaction number (流水号, starts `4203`) into **Transaction ID**, and presses **I have paid** (`POST /swaps/:id/proof`). The optional receipt image is stored as-is in the proof store; only its `sha256` is anchored on the swap event.



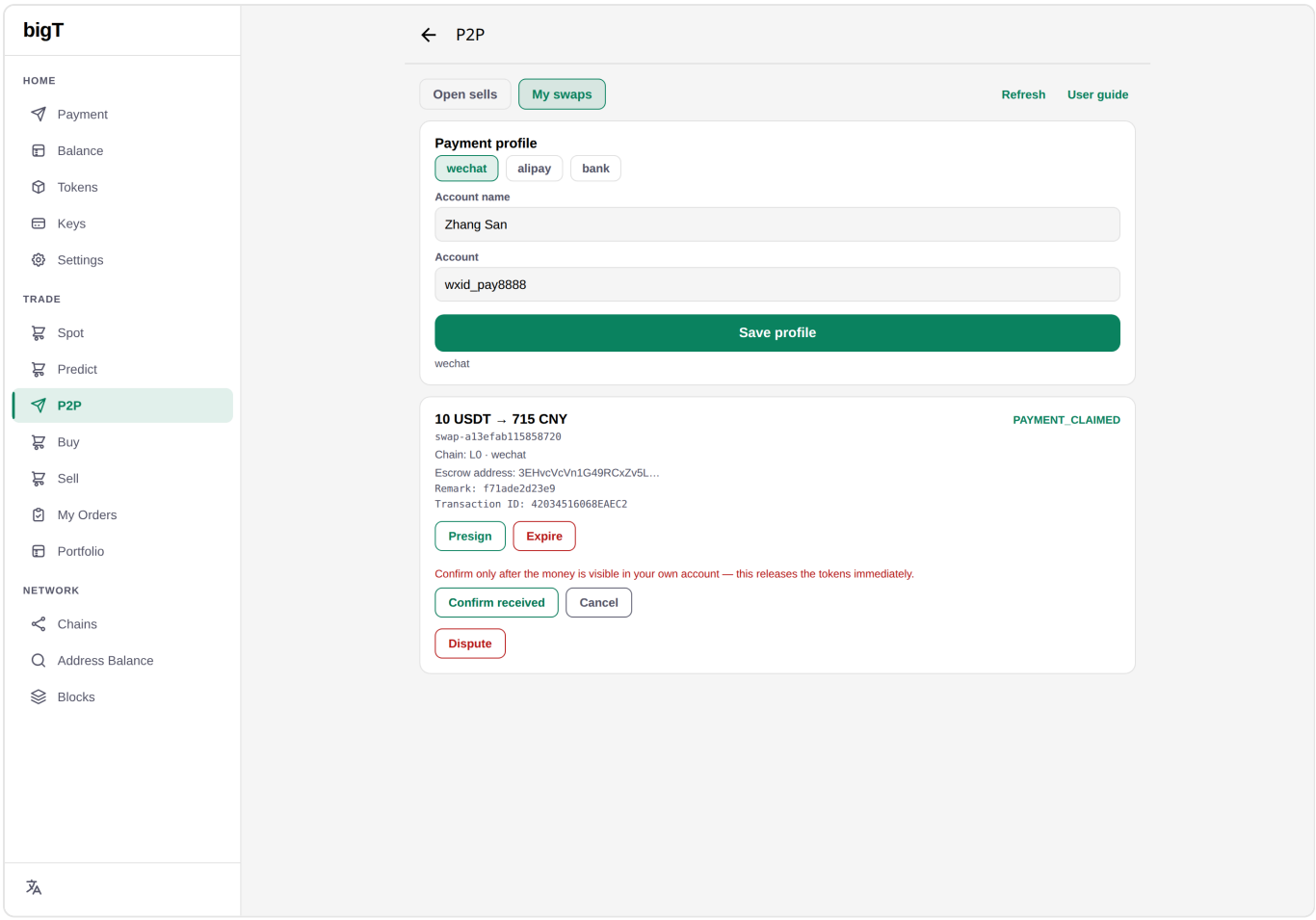
Status is now `PAYMENT_CLAIMED` — **the tokens are still locked.** This state exists purely as the buyer's claim.



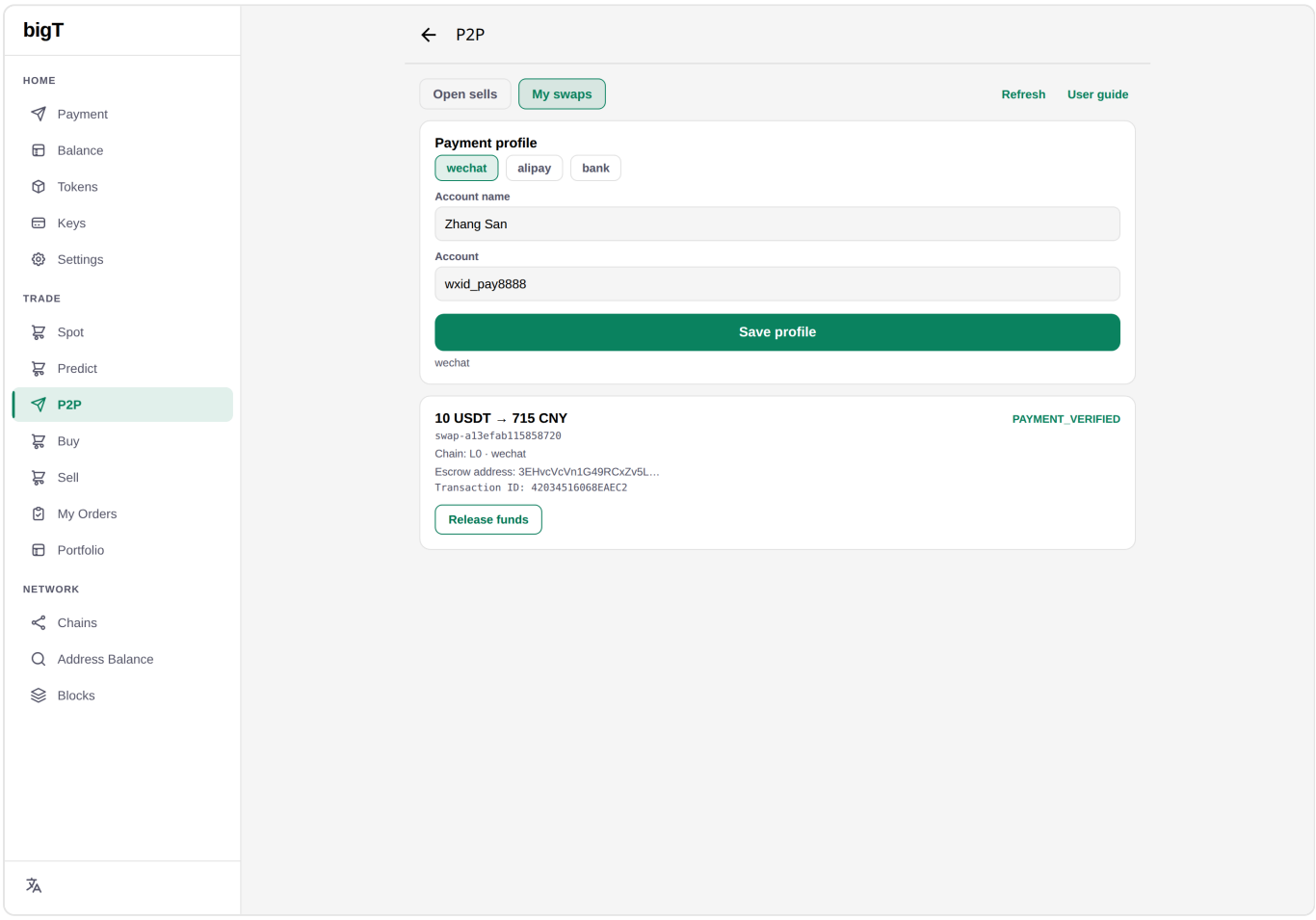
Step 7 — Seller verifies on their own statement, then confirms

The seller's card shows the buyer's `流水号` and the remark. The seller checks **amount + remark + time + 流水号 in their own WeChat account** (screenshots never count), and only then presses **Confirm received** — a seller-signed `POST /swaps/:id/confirm`. The red warning above the button is the whole safety story of the CNY rail:

> Confirm only after the money is visible in your own account — this releases the tokens immediately.



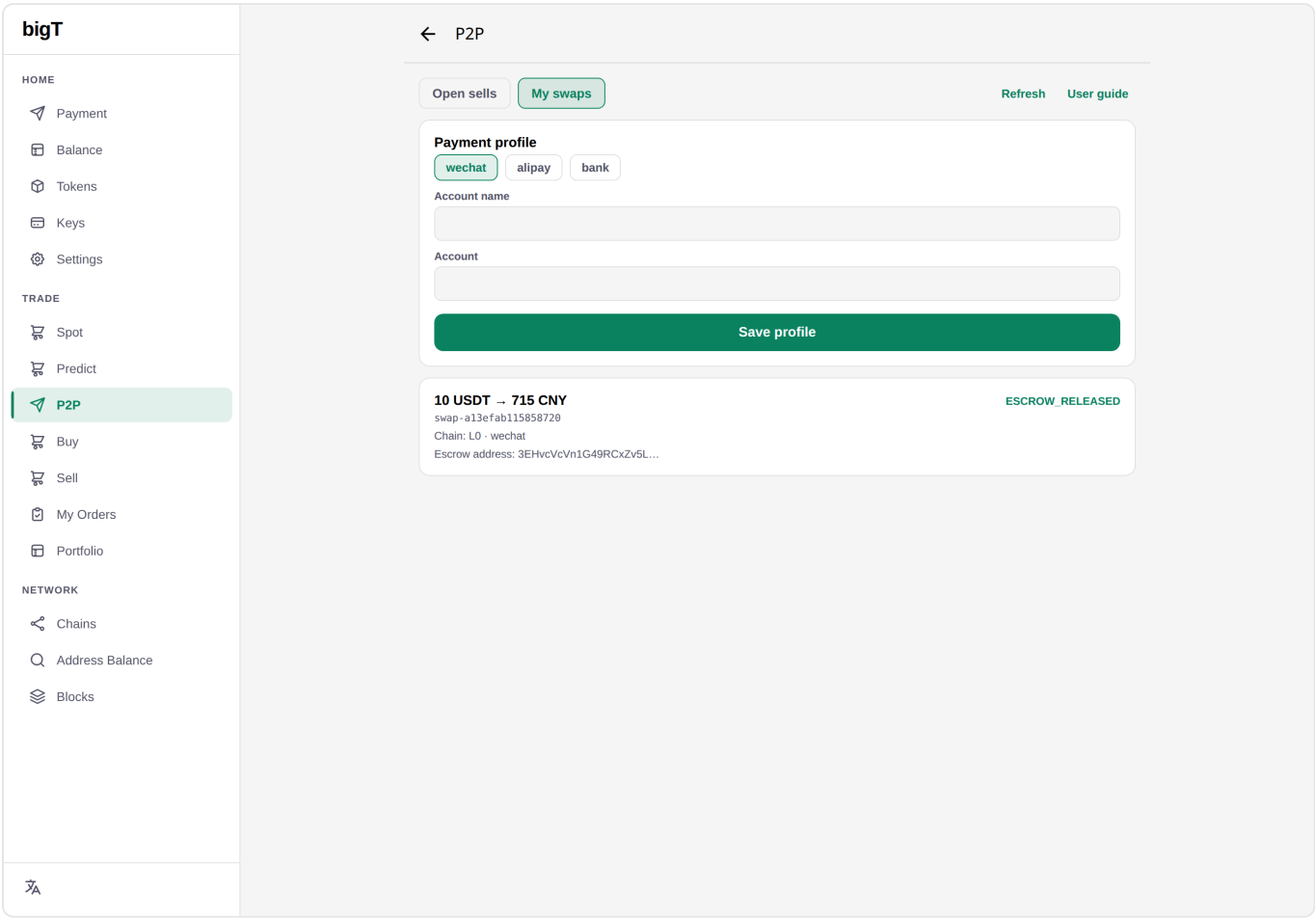
`PAYMENT_VERIFIED`:

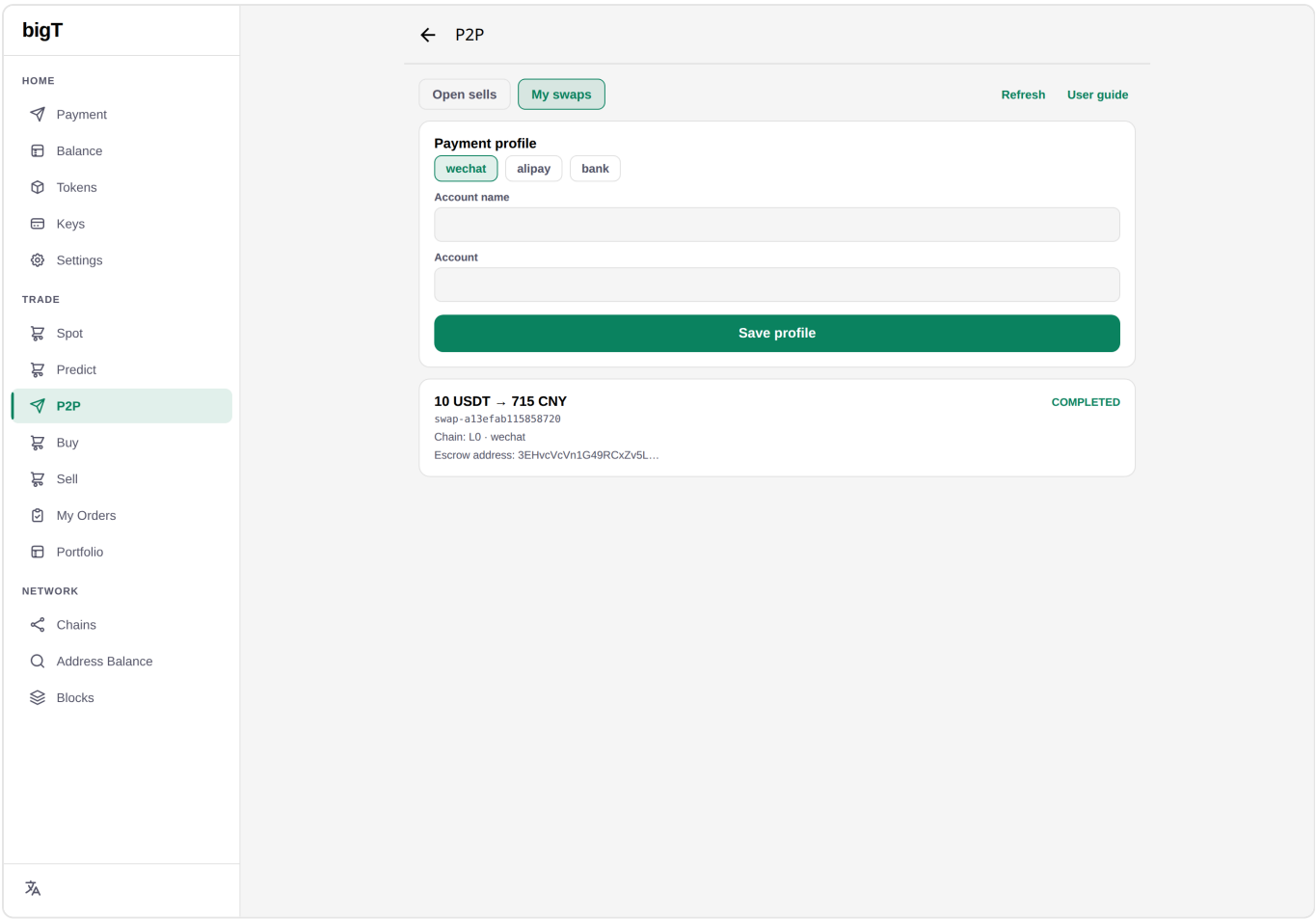


Step 8 — Engine releases the tokens, swap completes

`release` and `complete` are engine-signed actions — the wallet deliberately holds no key for them (§6.1). The capture script stands in for the operator / release job and signs both with the `SETTLEMENT_ENGINE_DID` key:

```
POST /swaps/swap-…/transitions   # did = engine, action=release, txHash=…
→ { "status": "ESCROW_RELEASED" }
POST /swaps/swap-…/transitions   # did = engine, action=complete
→ { "status": "COMPLETED" }
```

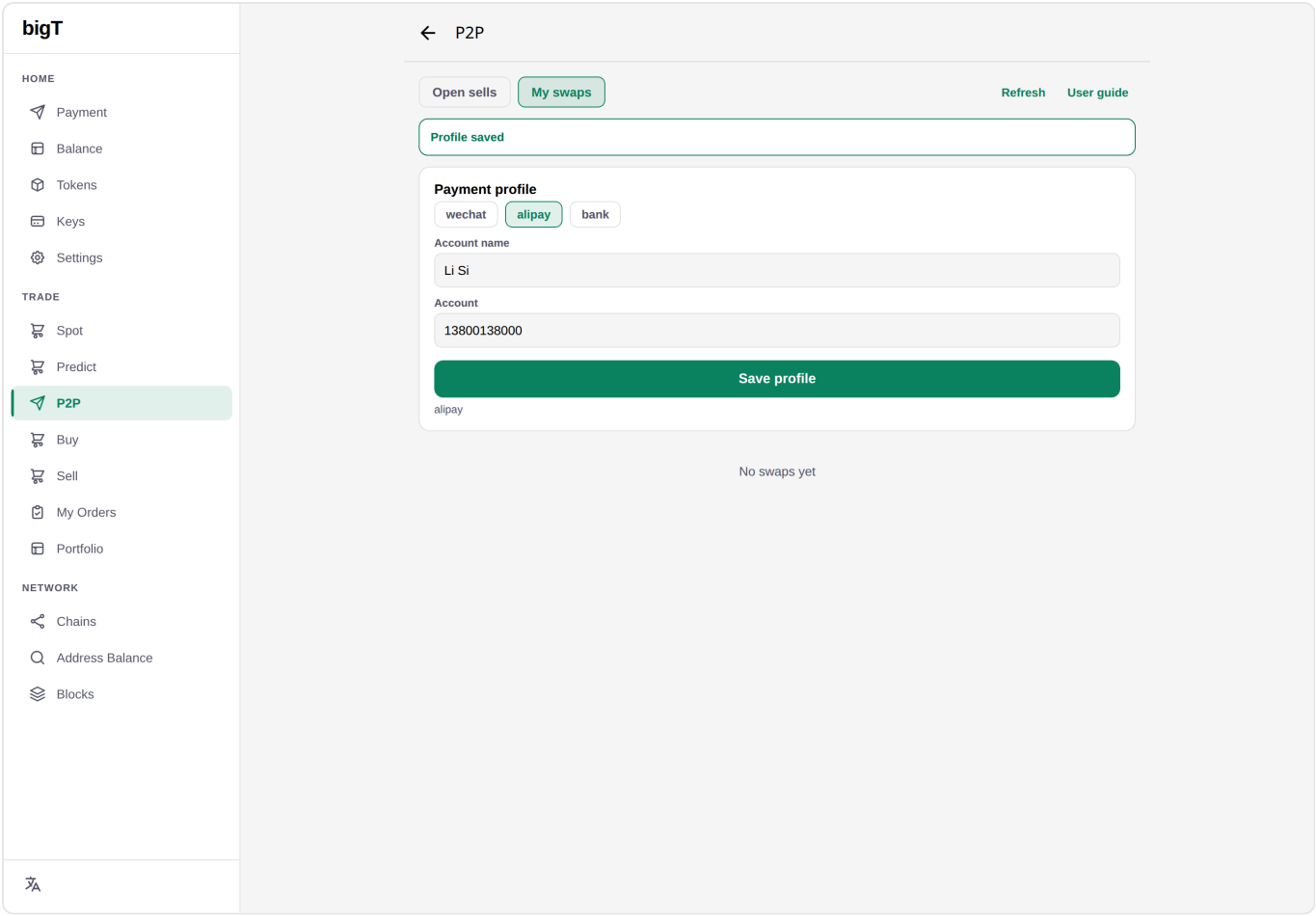




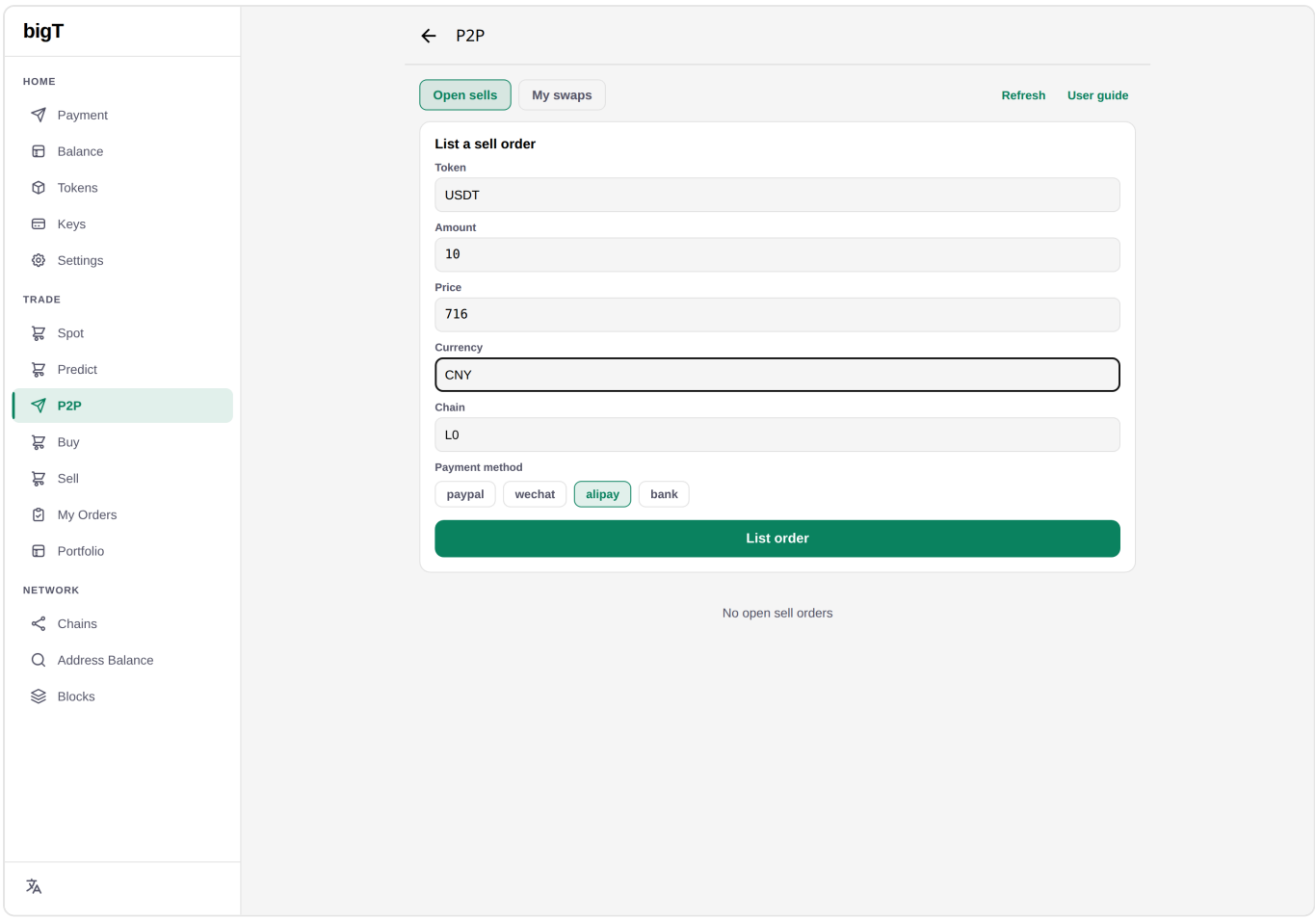
5. Walkthrough — Alipay (same shape, different rail)

The Alipay leg is the **same state machine** with `wantRail=alipay`: the seller saves an Alipay collection profile (account = 手机号 / Alipay logon), lists USDT for CNY on the alipay chip, and the buyer pays in their own Alipay app. The 流水号 is Alipay's 28-digit trade number (starts `20`). Fresh wallets are used so the screenshots stand alone.

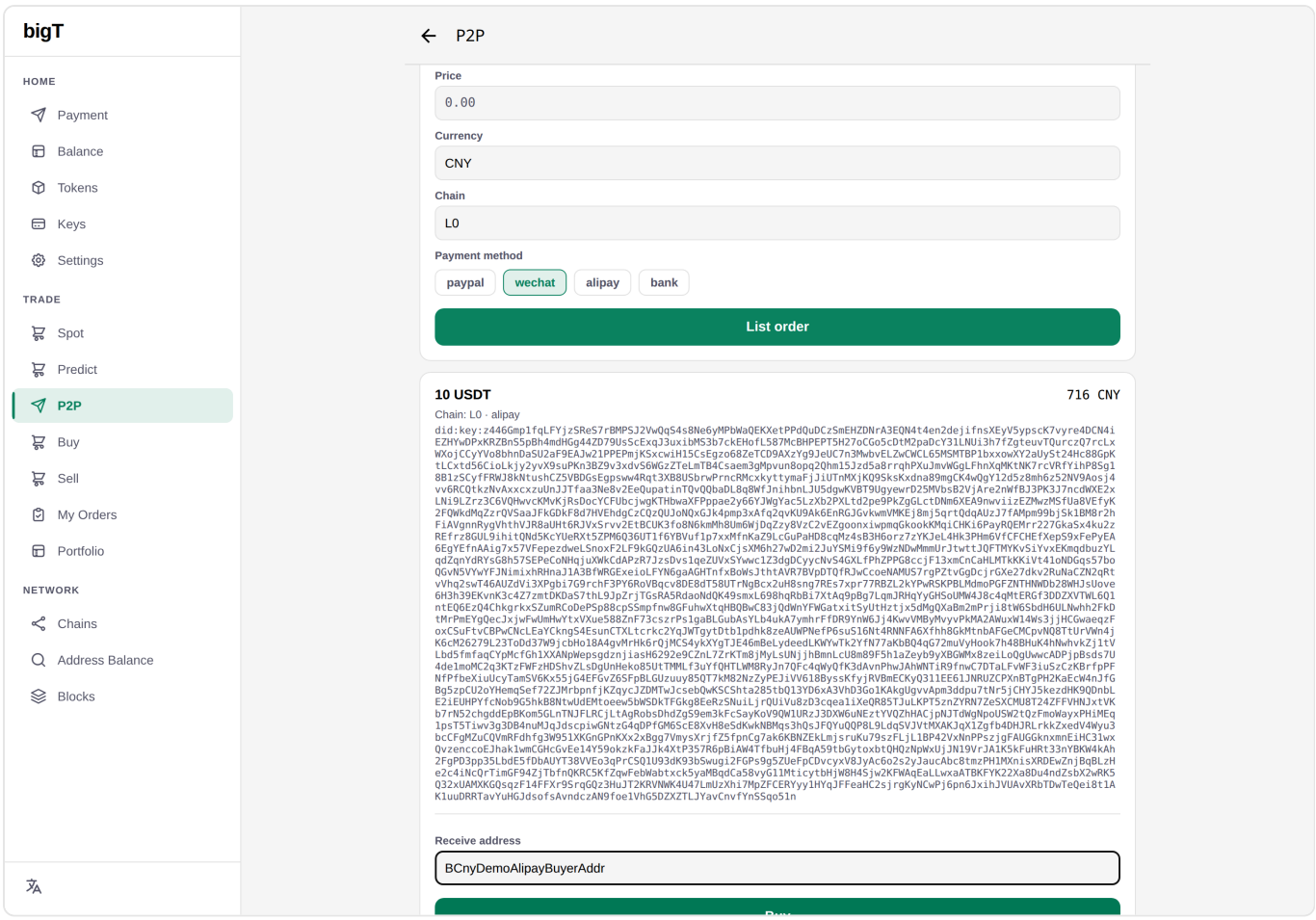
Step 1 — Seller saves the Alipay profile



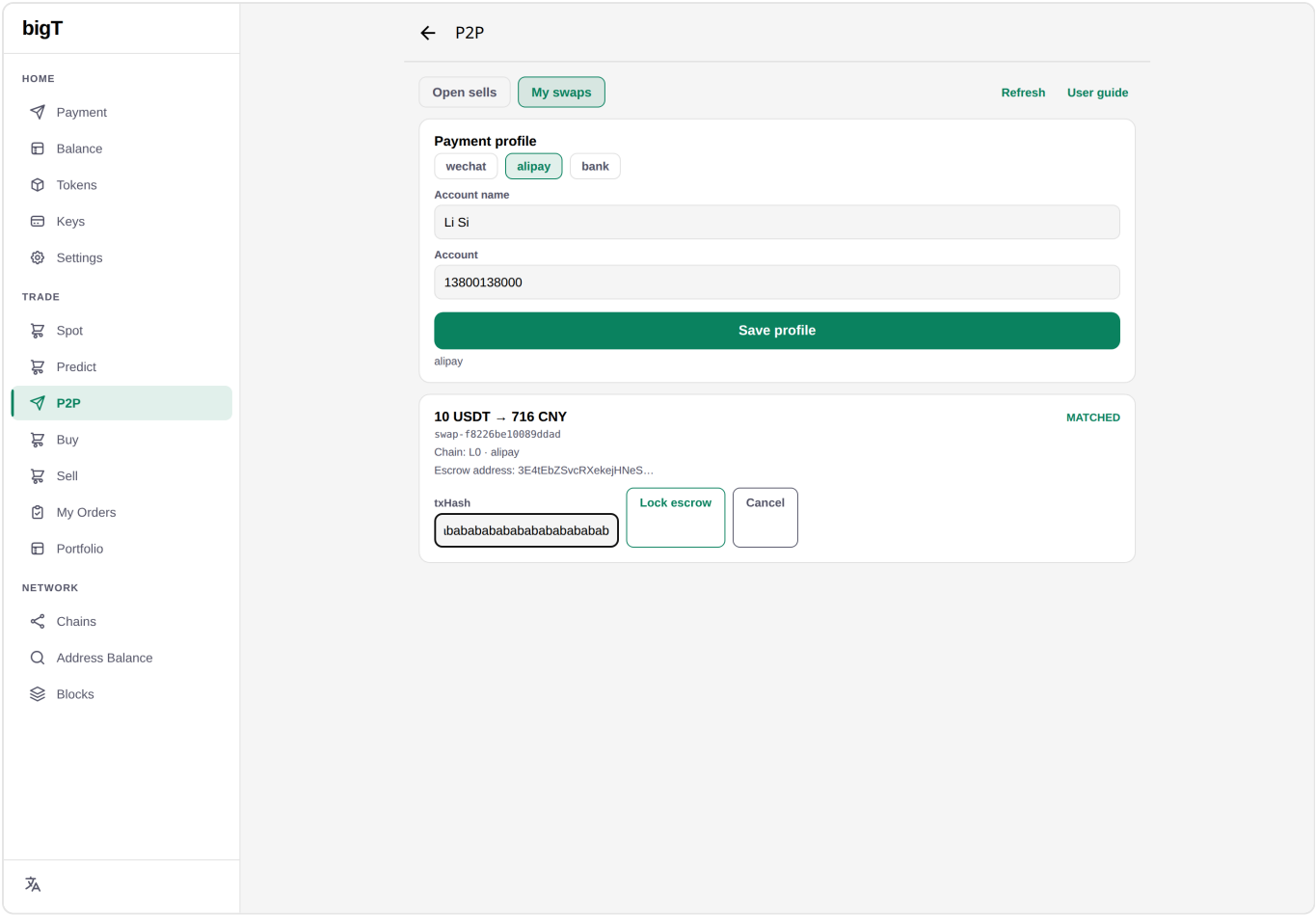
Step 2 — Seller lists 10 USDT for 716 CNY on the alipay rail

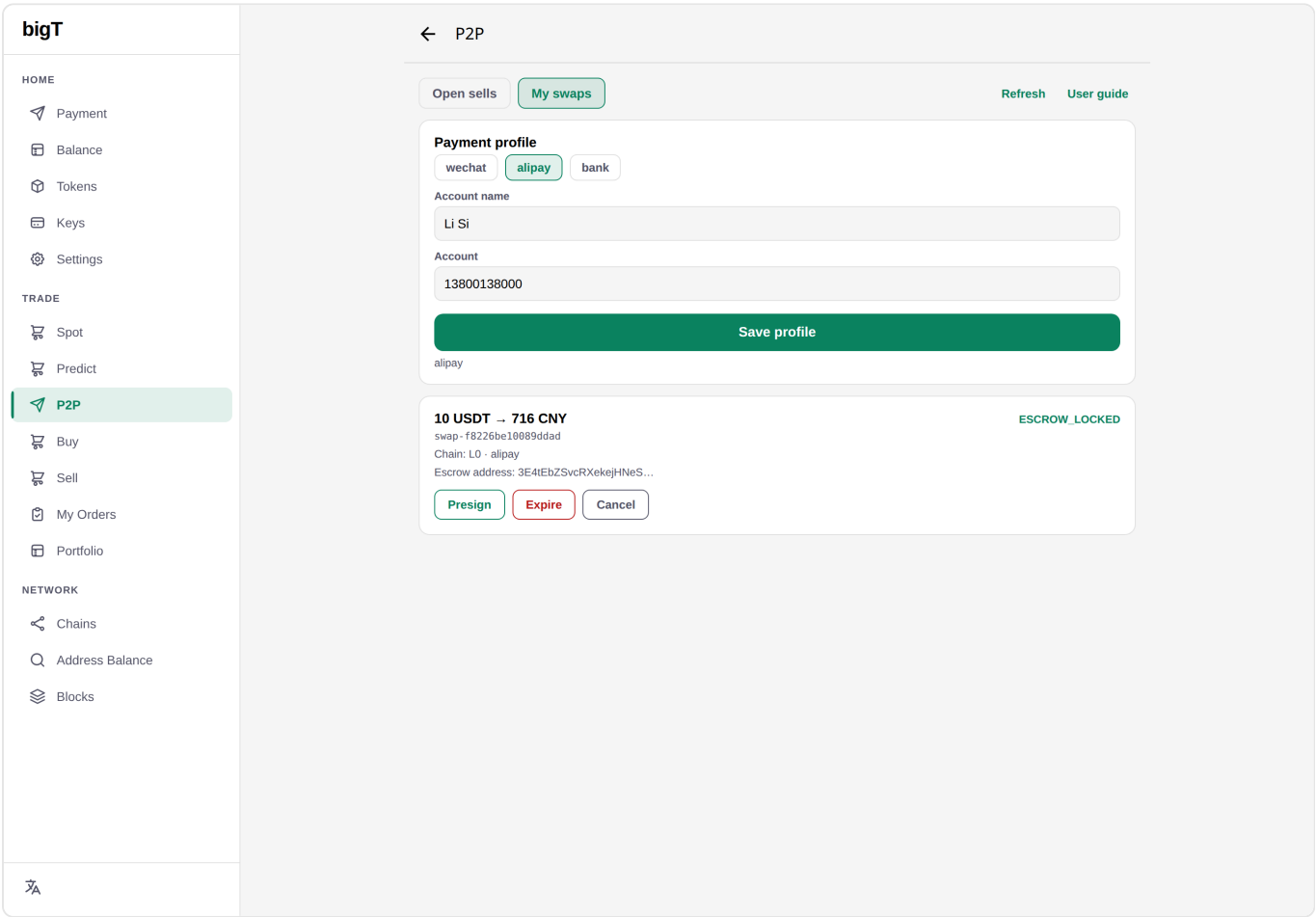


Step 3 — Buyer matches



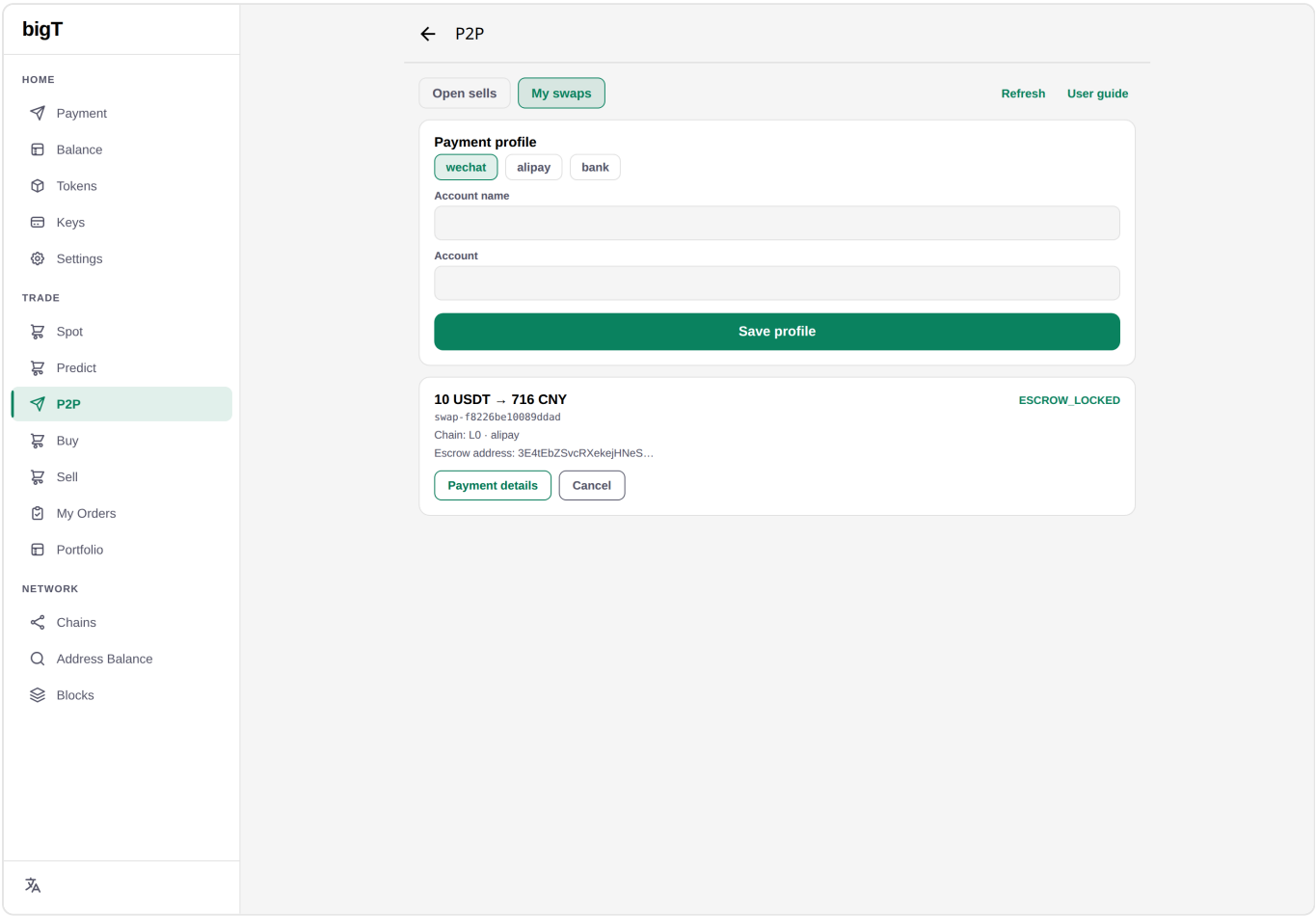
Step 4 — Seller locks the escrow

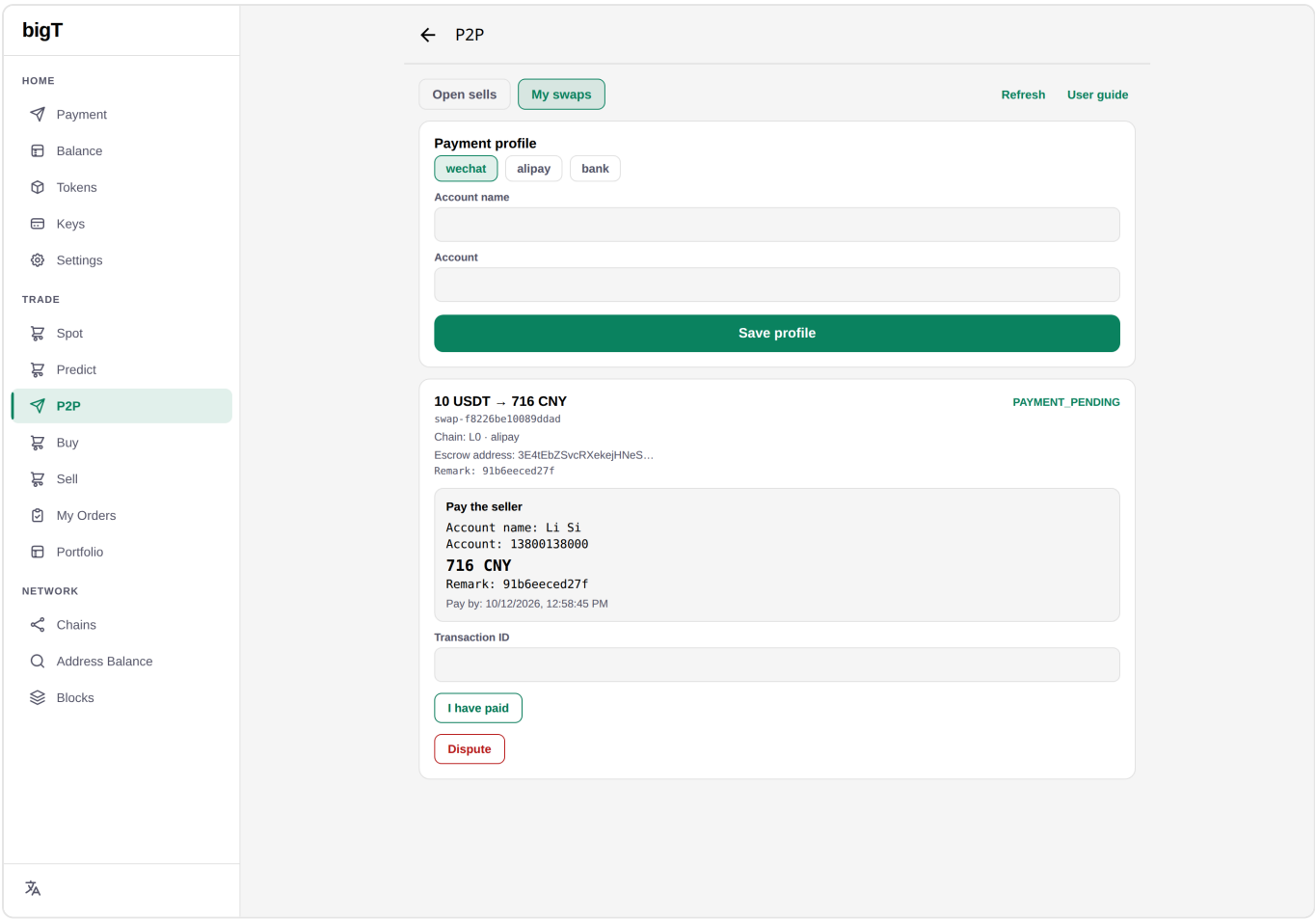




Step 5 — Buyer pulls the Alipay payment instructions

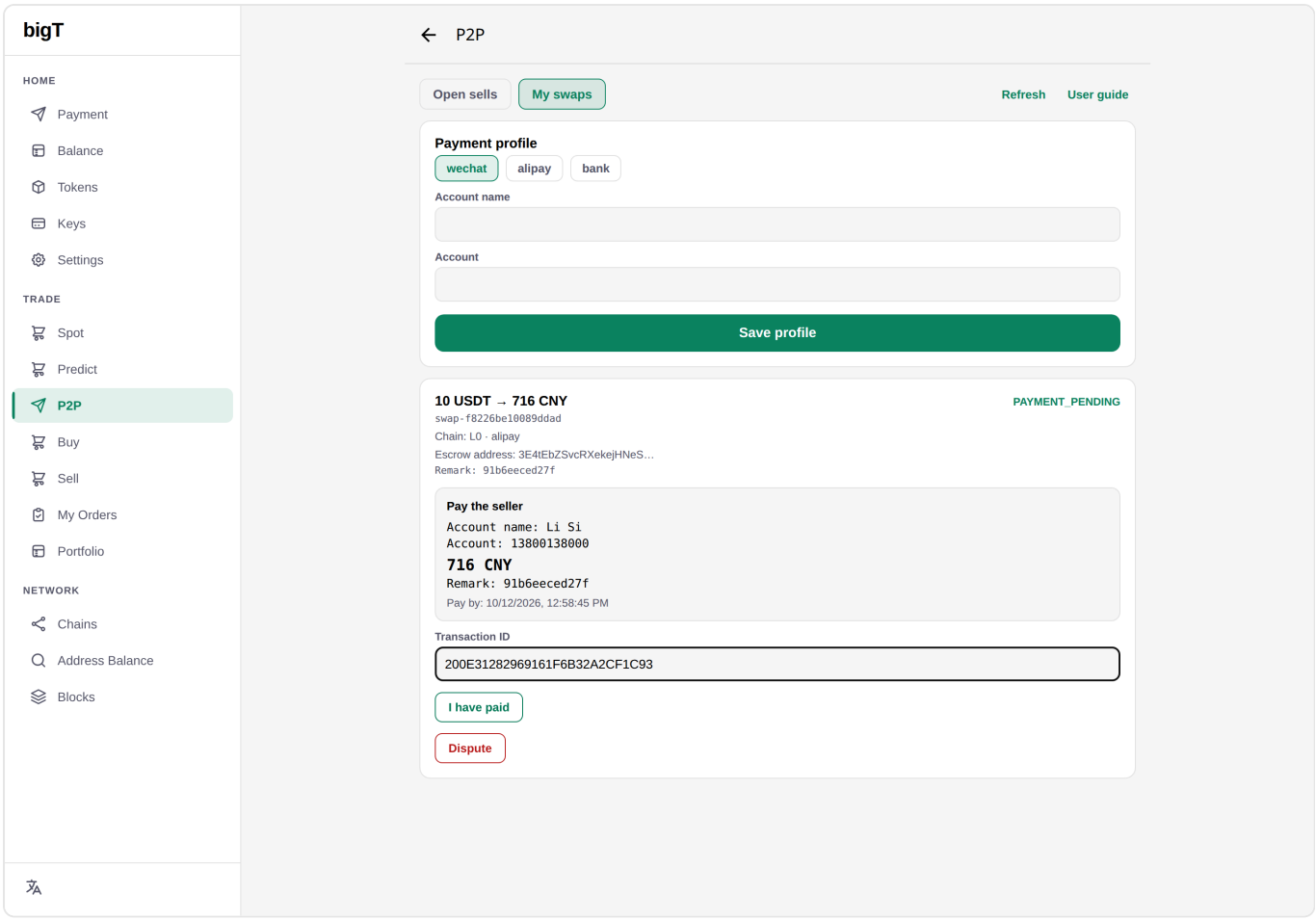

```
{ "method": "alipay", "rail": "alipay", "accountName": "Li Si",
  "account": "13800138000", "amount": "716", "currency": "CNY",
  "remark": "91b6eeced27f", "payBy": … }
```

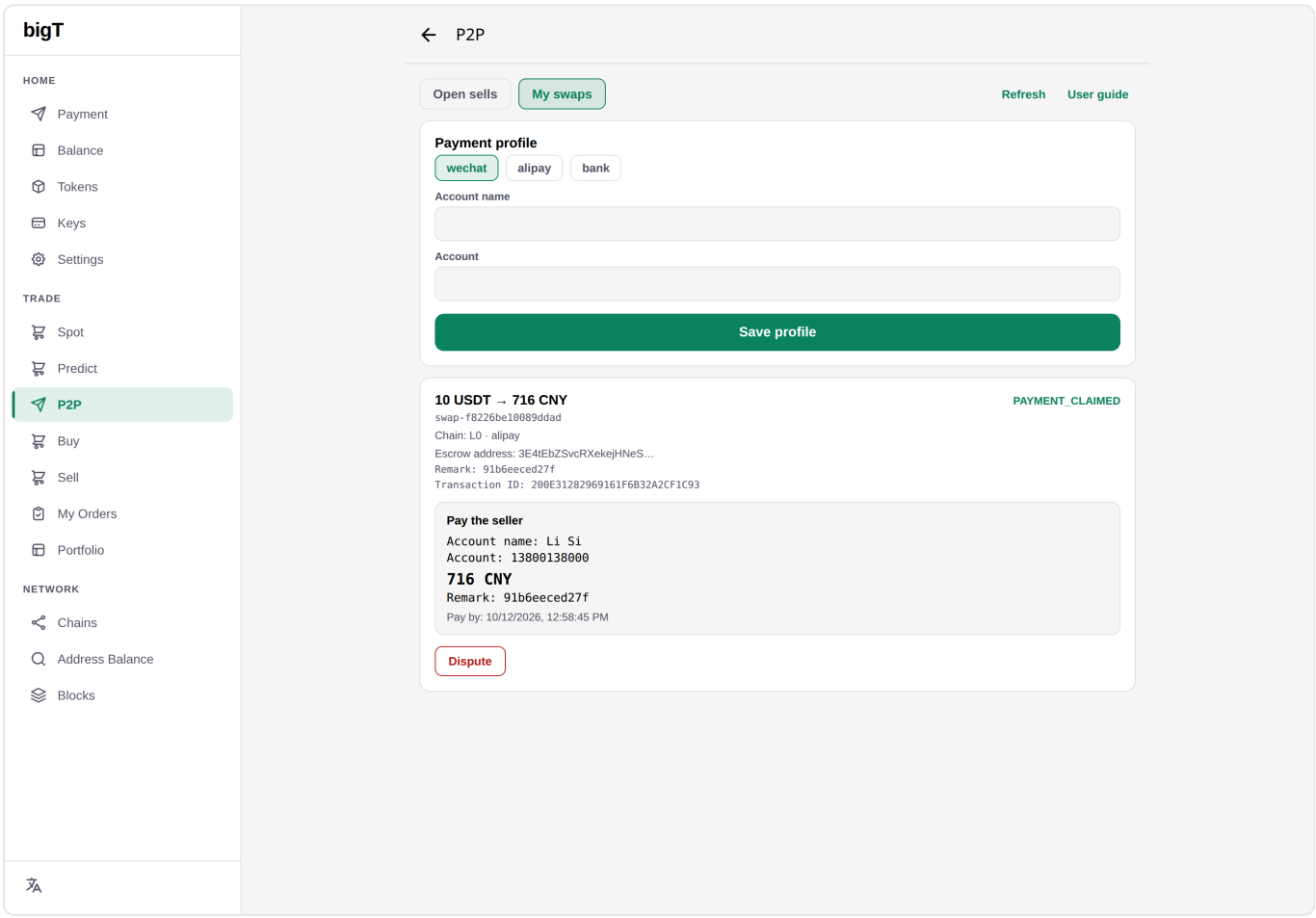





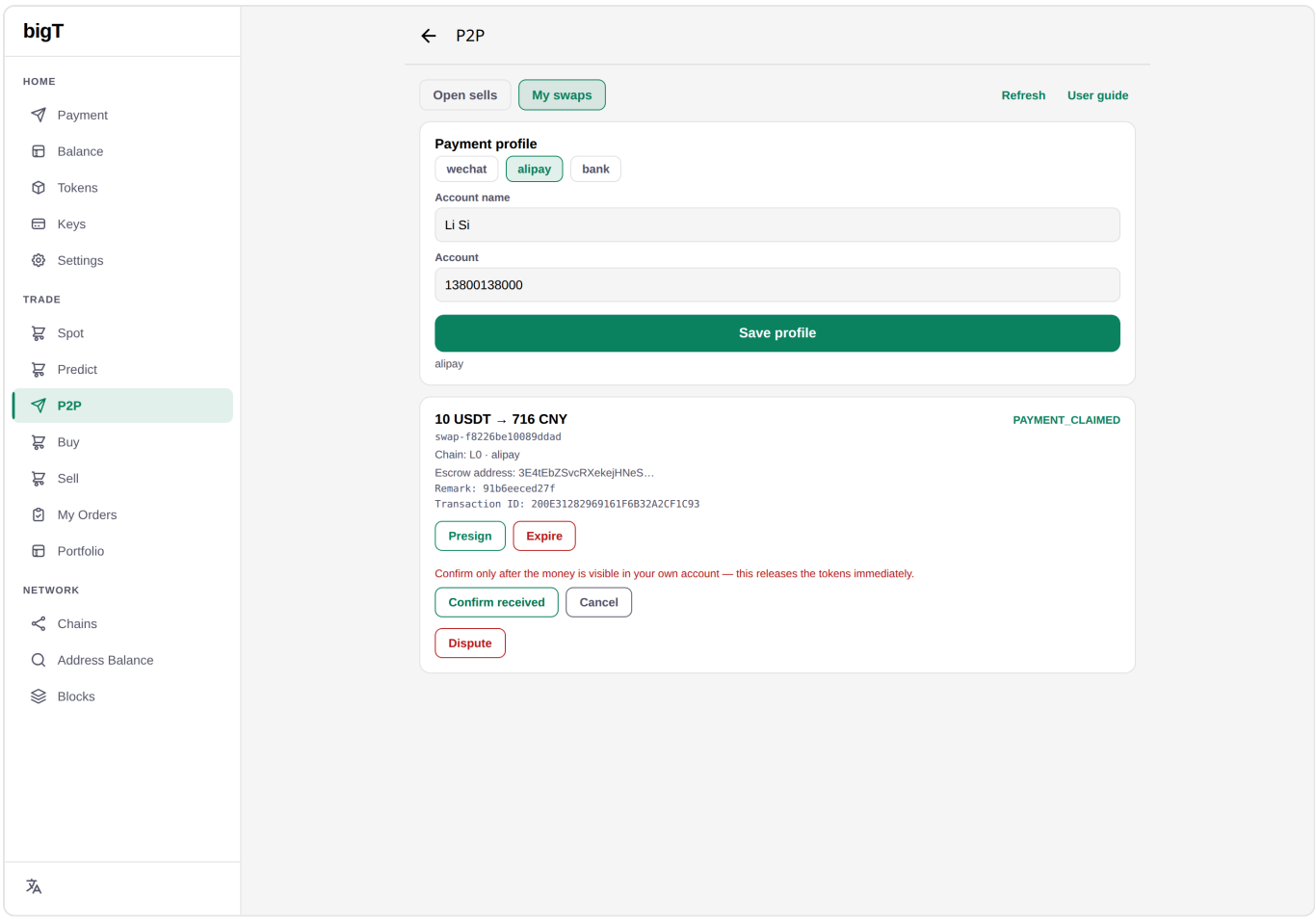
Step 6 — Buyer pays in Alipay and claims with the trade number

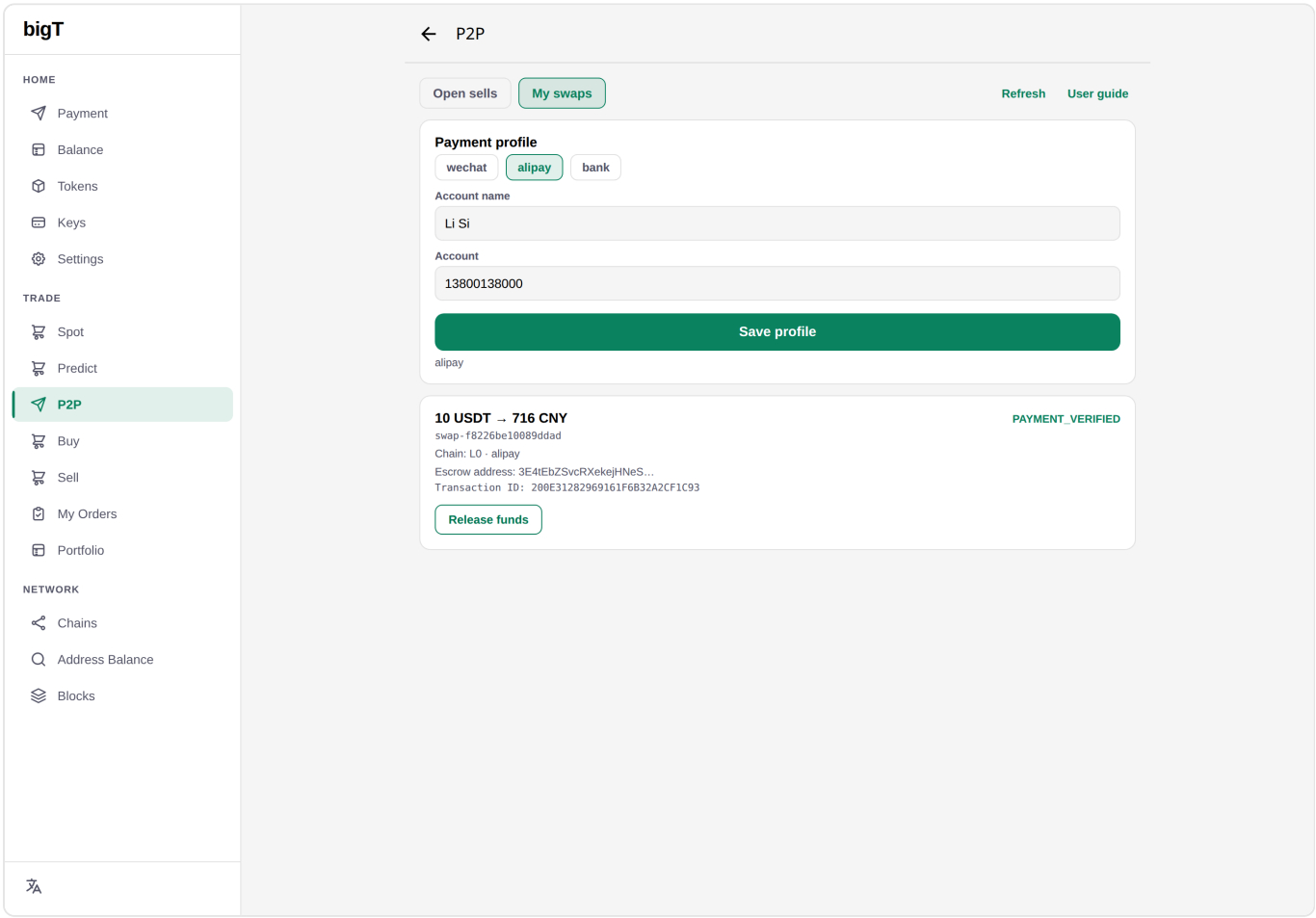
The buyer transfers `716 CNY` in Alipay, pastes the 28-digit trade number (`200E31282969161F6B32A2CF1C93` in this run), and presses **I have paid**.


bigT
HOME
Payment
Balance
Tokens
Keys
Settings
TRADE
Spot
Predict
P2P
Buy
Sell
My Orders
Portfolio
NETWORK
Chains
Address Balance
Blocks
P2P
Open sells
My swaps
Refresh
User guide
Payment profile
wechat
alipay
bank
Account name
Account
Save profile
10 USDT → 716 CNY
PAYMENT_PENDING
swap-f8226be10089ddad
Chain: L0 · alipay
Escrow address: 3E4tEbZSvcRXekejHNeS…
Remark: 91b6eeced27f
Pay the seller
Account name: Li Si
Account: 13800138000
716 CNY
Remark: 91b6eeced27f
Pay by: 10/12/2026, 12:58:45 PM
Transaction ID
200E31282969161F6B32A2CF1C93
I have paid
Dispute

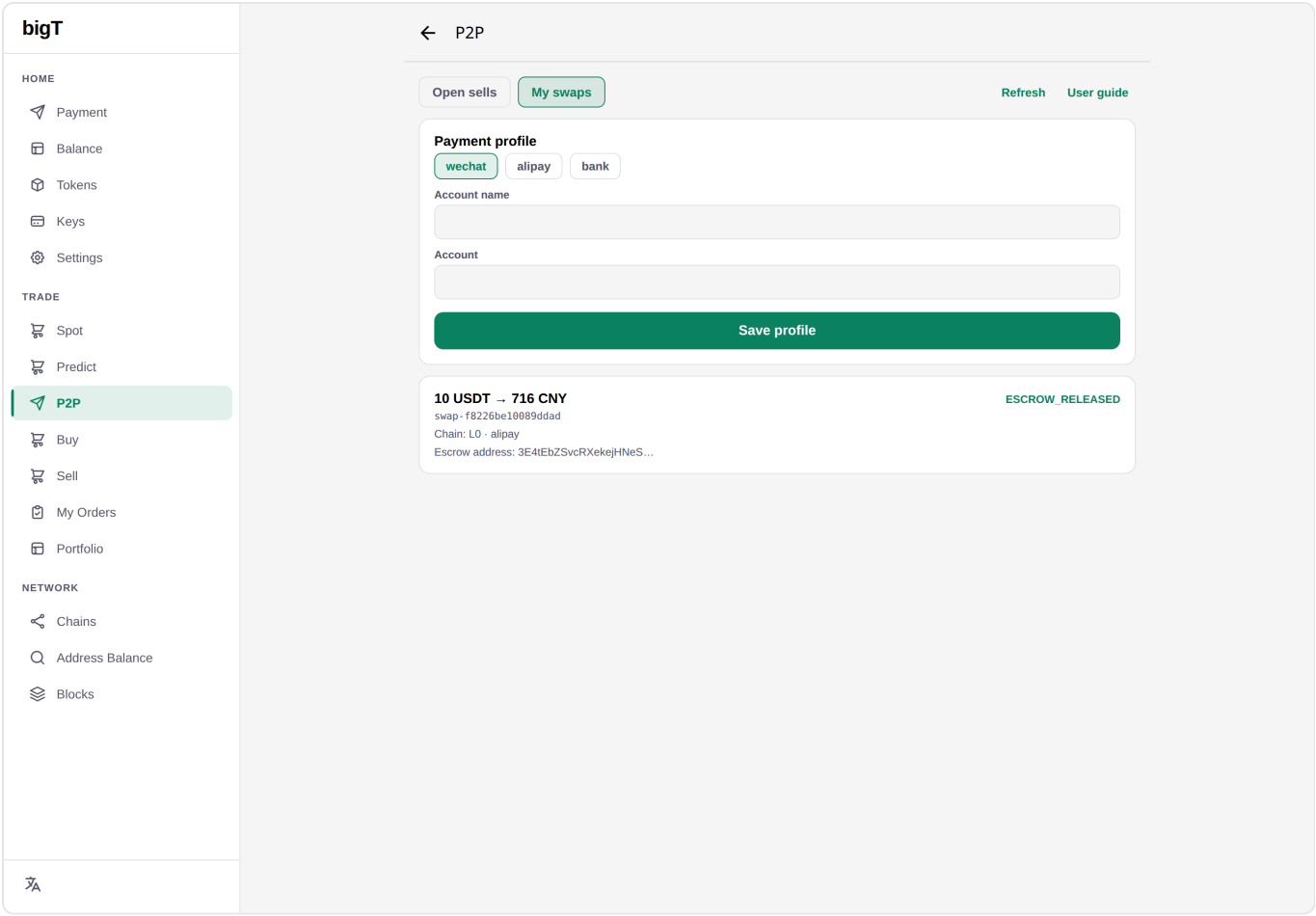

bigT
HOME
Payment
Balance
Tokens
Keys
Settings
TRADE
Spot
Predict
P2P
Buy
Sell
My Orders
Portfolio
NETWORK
Chains
Address Balance
Blocks
P2P
Open sells
My swaps
Refresh
User guide
Payment profile
wechat
alipay
bank
Account name
Account
Save profile
10 USDT → 716 CNY
PAYMENT_CLAIMED
swap-f8226be10089ddad
Chain: L0 · alipay
Escrow address: 3E4tEbZSvcRXekejHNeS…
Remark: 91b6eeced27f
Transaction ID: 200E31282969161F6B32A2CF1C93
Pay the seller
Account name: Li Si
Account: 13800138000
716 CNY
Remark: 91b6eeced27f
Pay by: 10/12/2026, 12:58:45 PM
Dispute

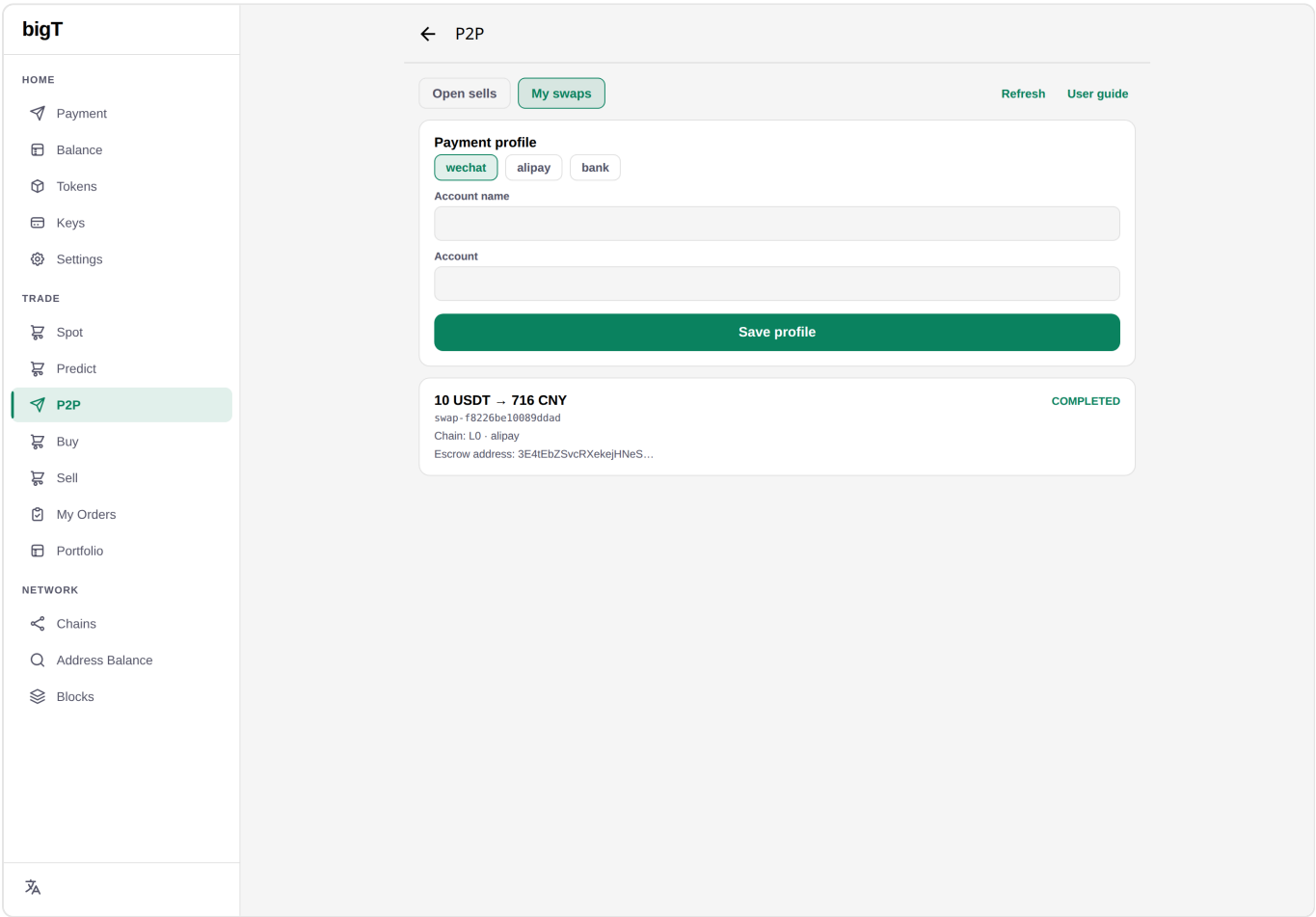
Step 7 — Seller verifies in Alipay, then confirms





Step 8 — Release + complete (engine-signed)

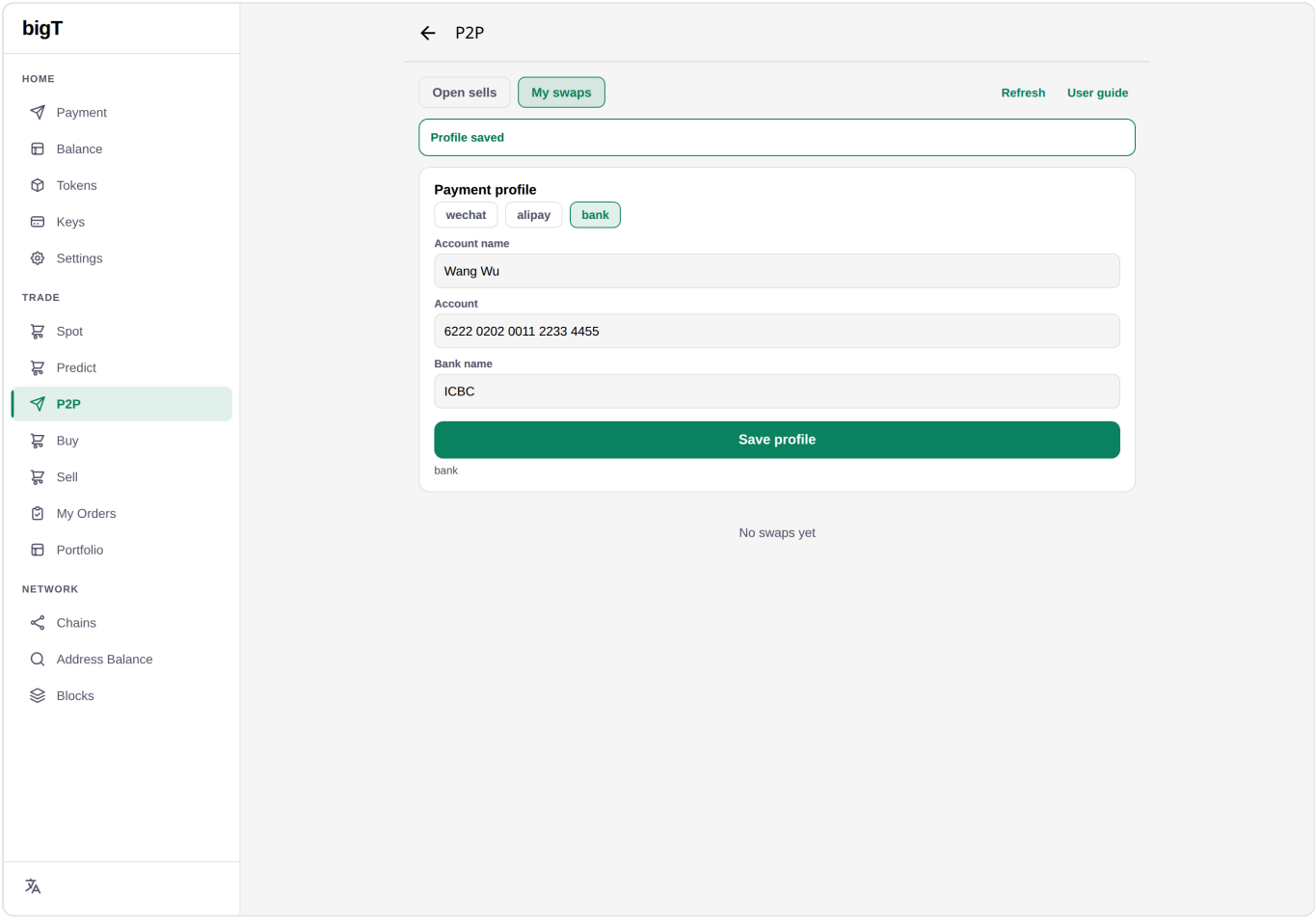




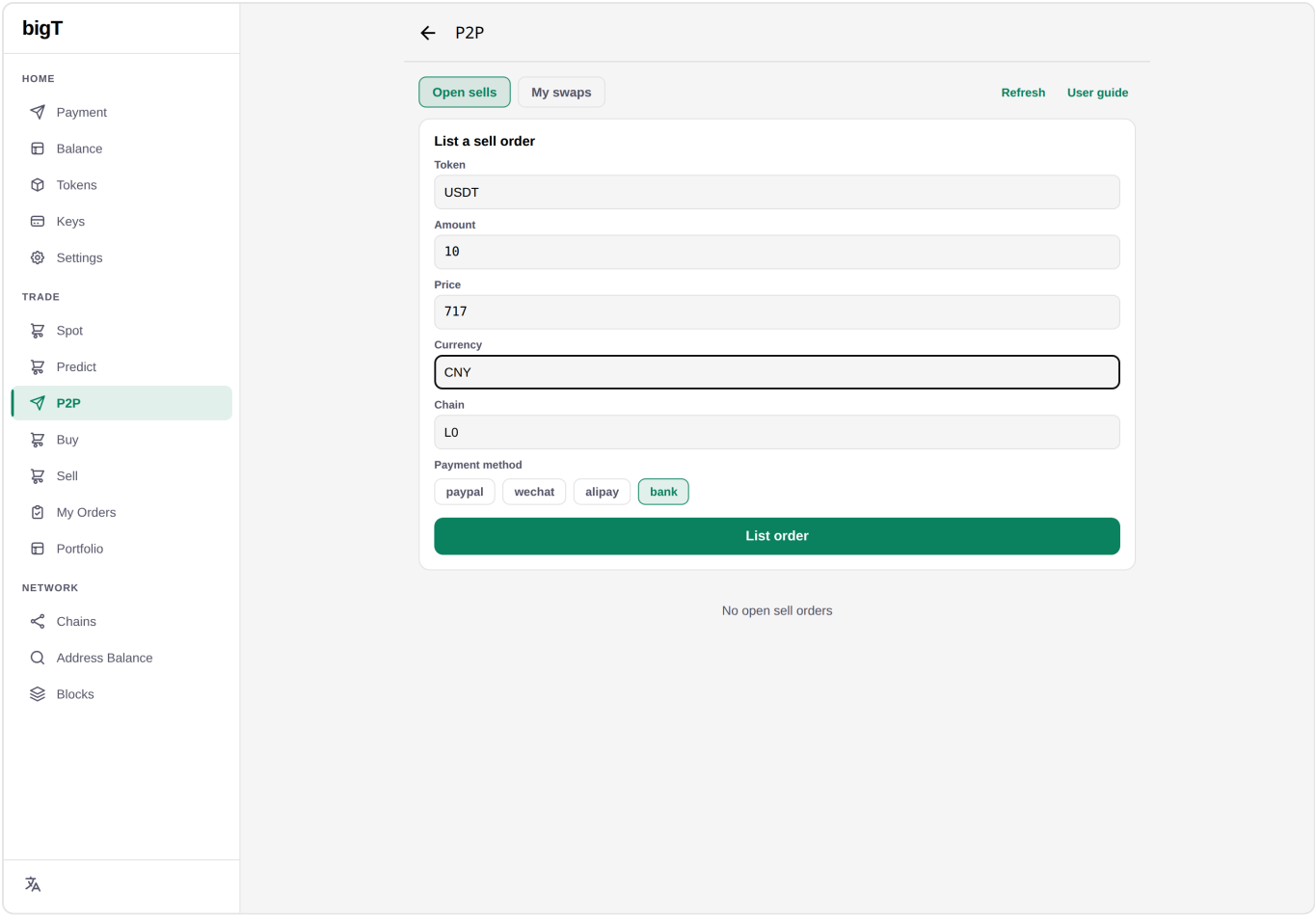
6. Walkthrough — Bank transfer (same shape, bank card)

The bank leg is again the **same state machine** with `wantRail=bank`. The collection profile carries one extra field — the **bank name** — which the instructions panel shows next to the card number. The buyer's 流水号 is the bank's 16-digit transfer reference (starts `3029` in this run).

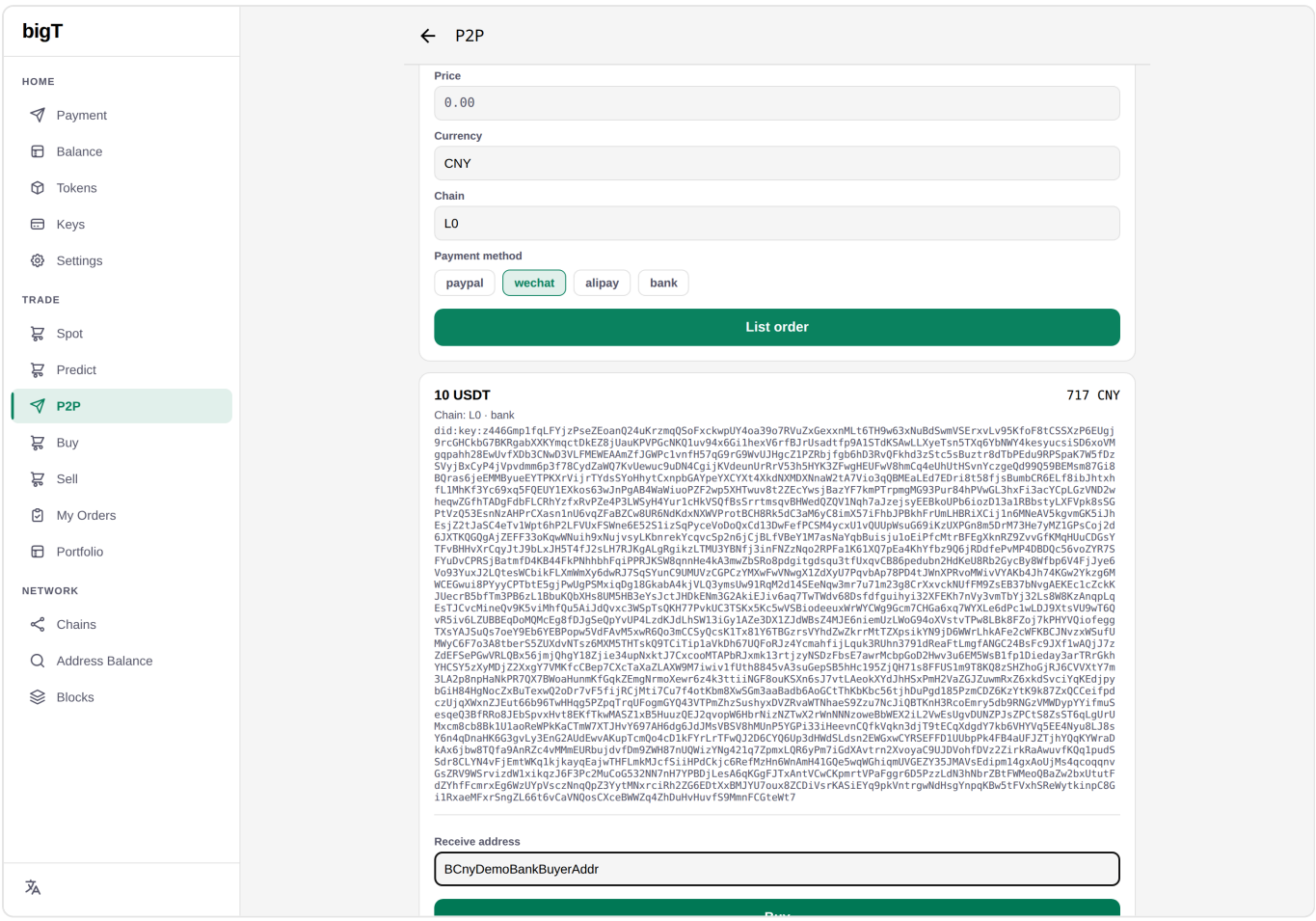
Step 1 — Seller saves the bank profile (name + card + bank)

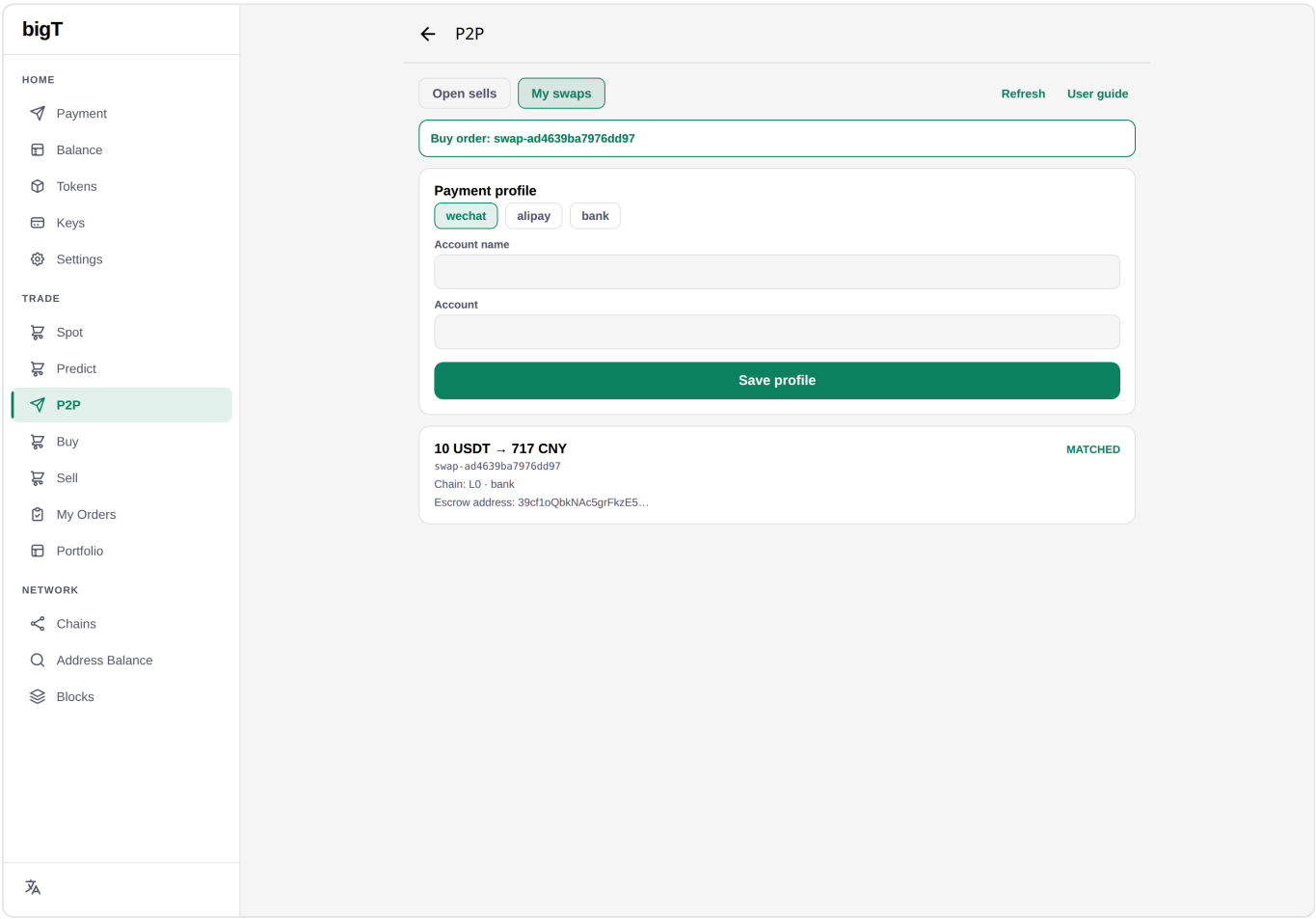


Step 2 — Seller lists 10 USDT for 717 CNY on the bank rail

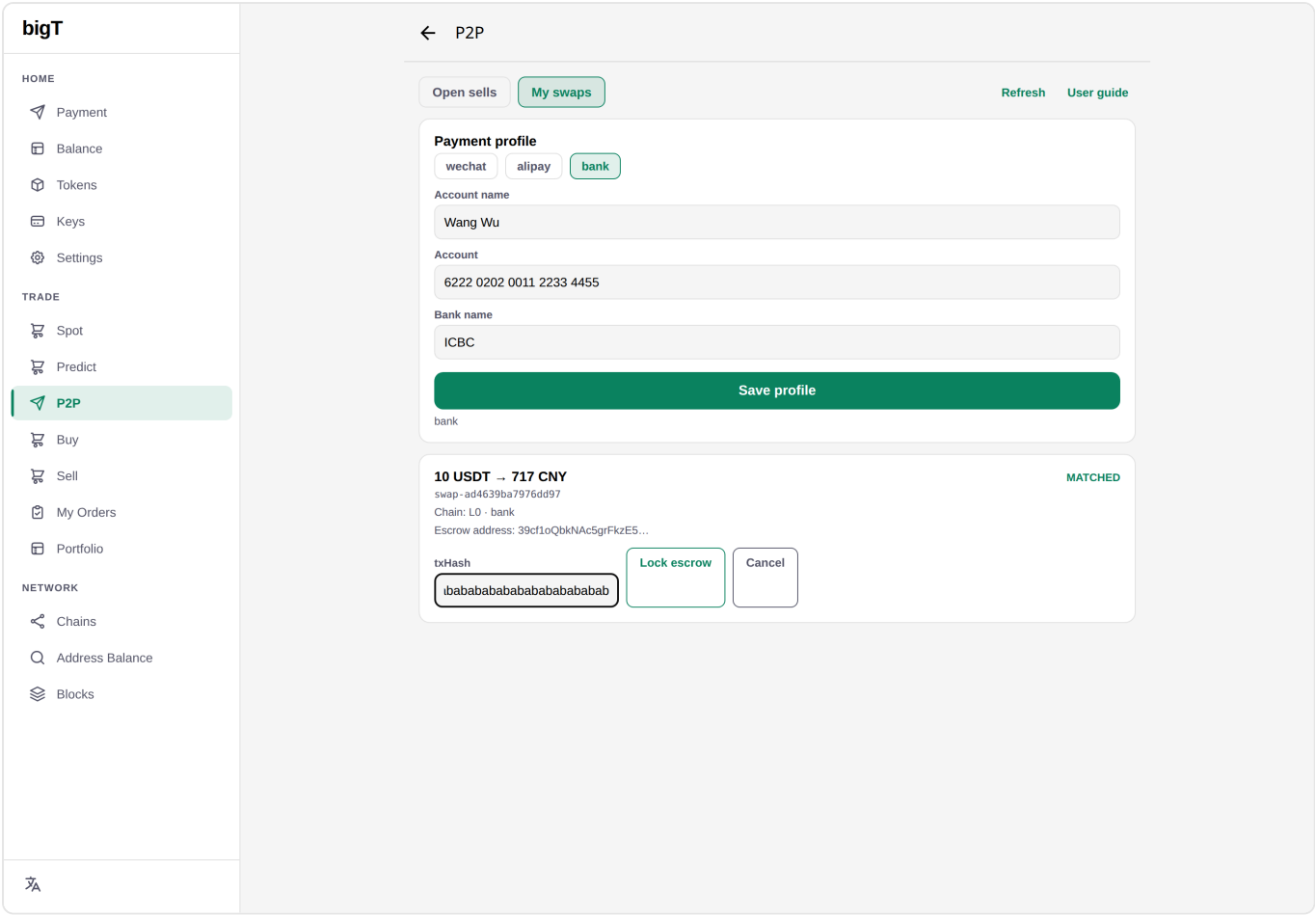


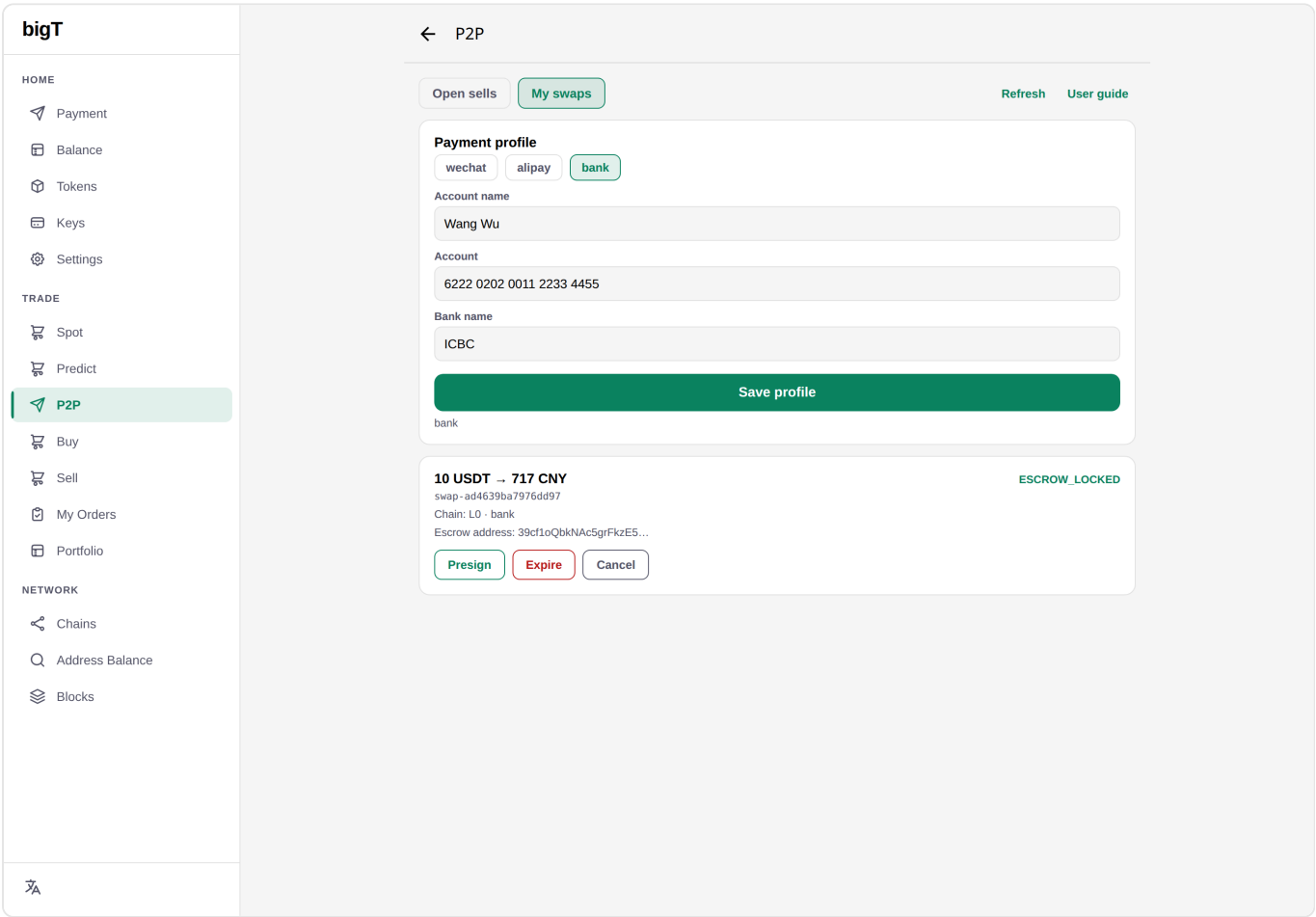
Step 3 — Buyer matches (only a receive address)





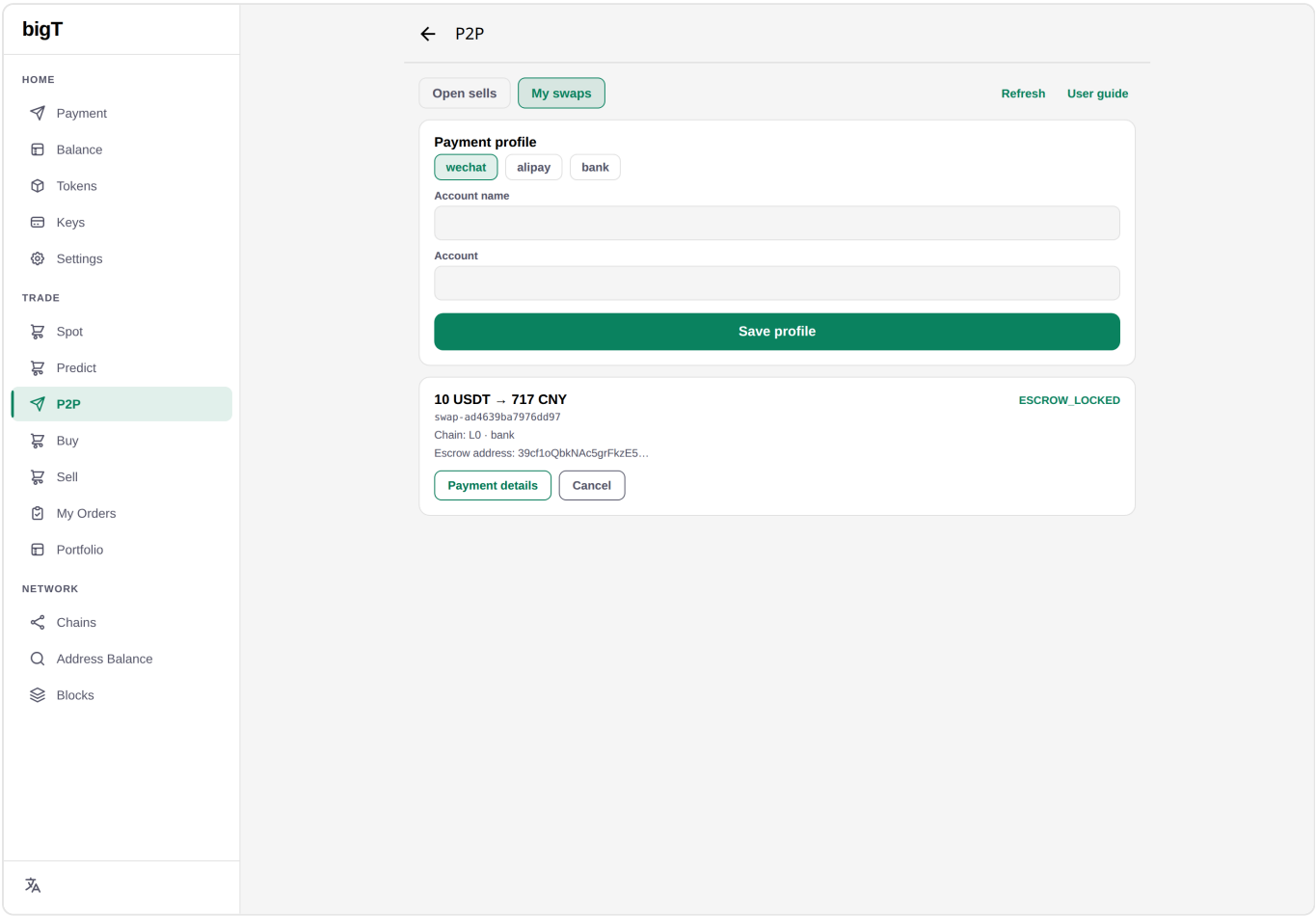
Step 4 — Seller locks the escrow

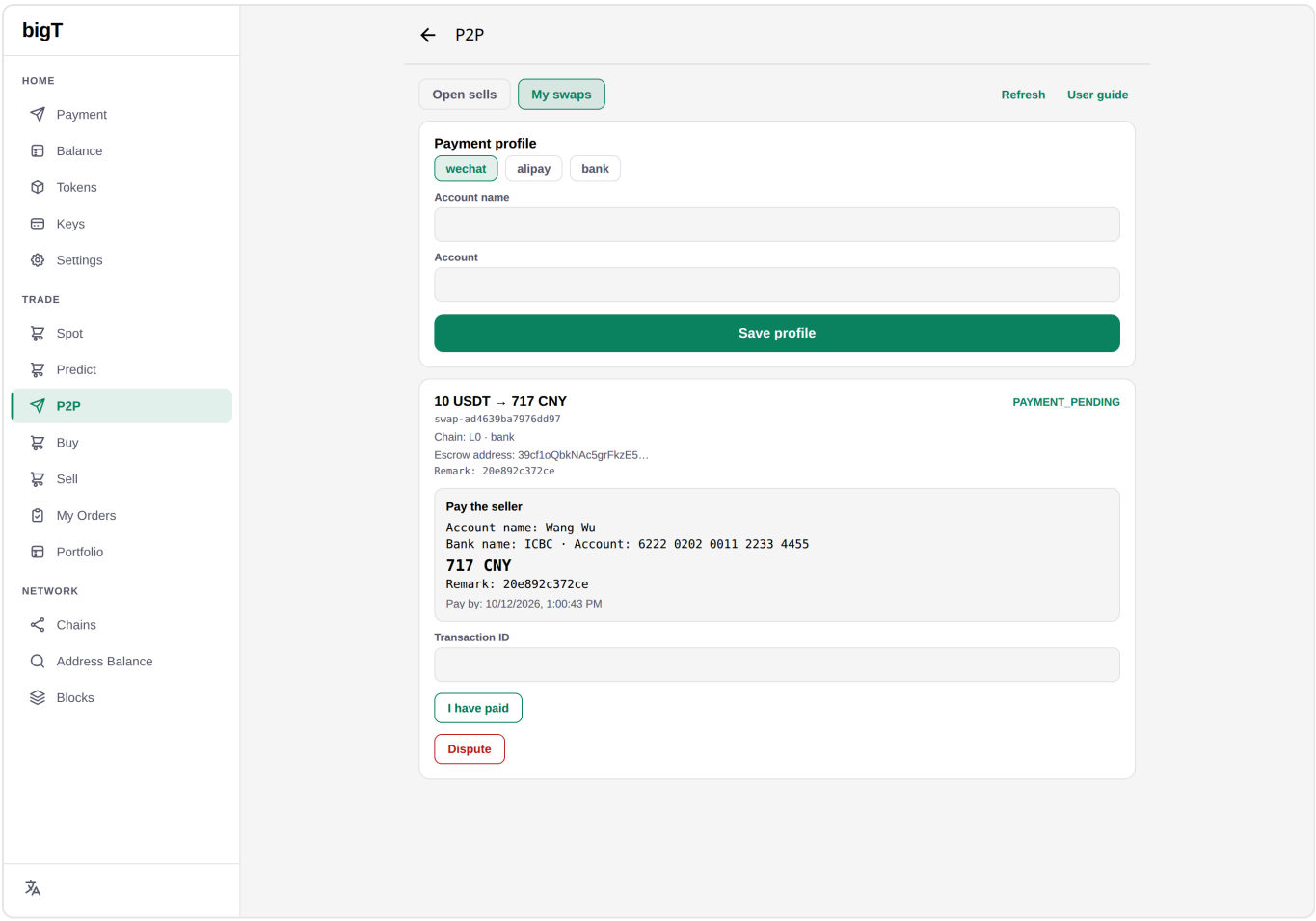




Step 5 — Buyer pulls the bank payment instructions

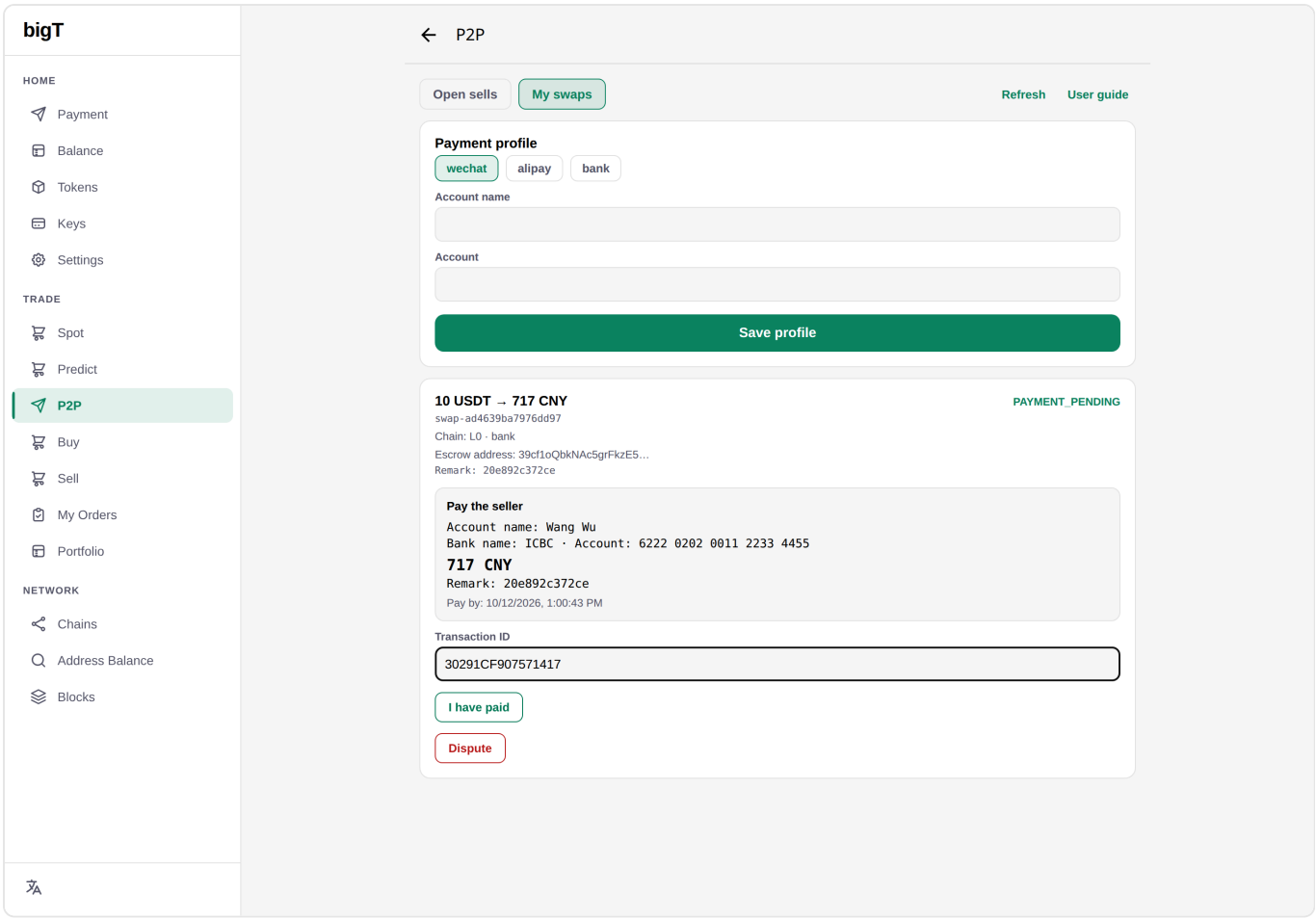

```
{ "method": "bank", "rail": "bank", "accountName": "Wang Wu",
  "bankName": "ICBC", "account": "6222 0202 0011 2233 4455",
  "amount": "717", "currency": "CNY",
  "remark": "20e892c372ce", "payBy": … }
```

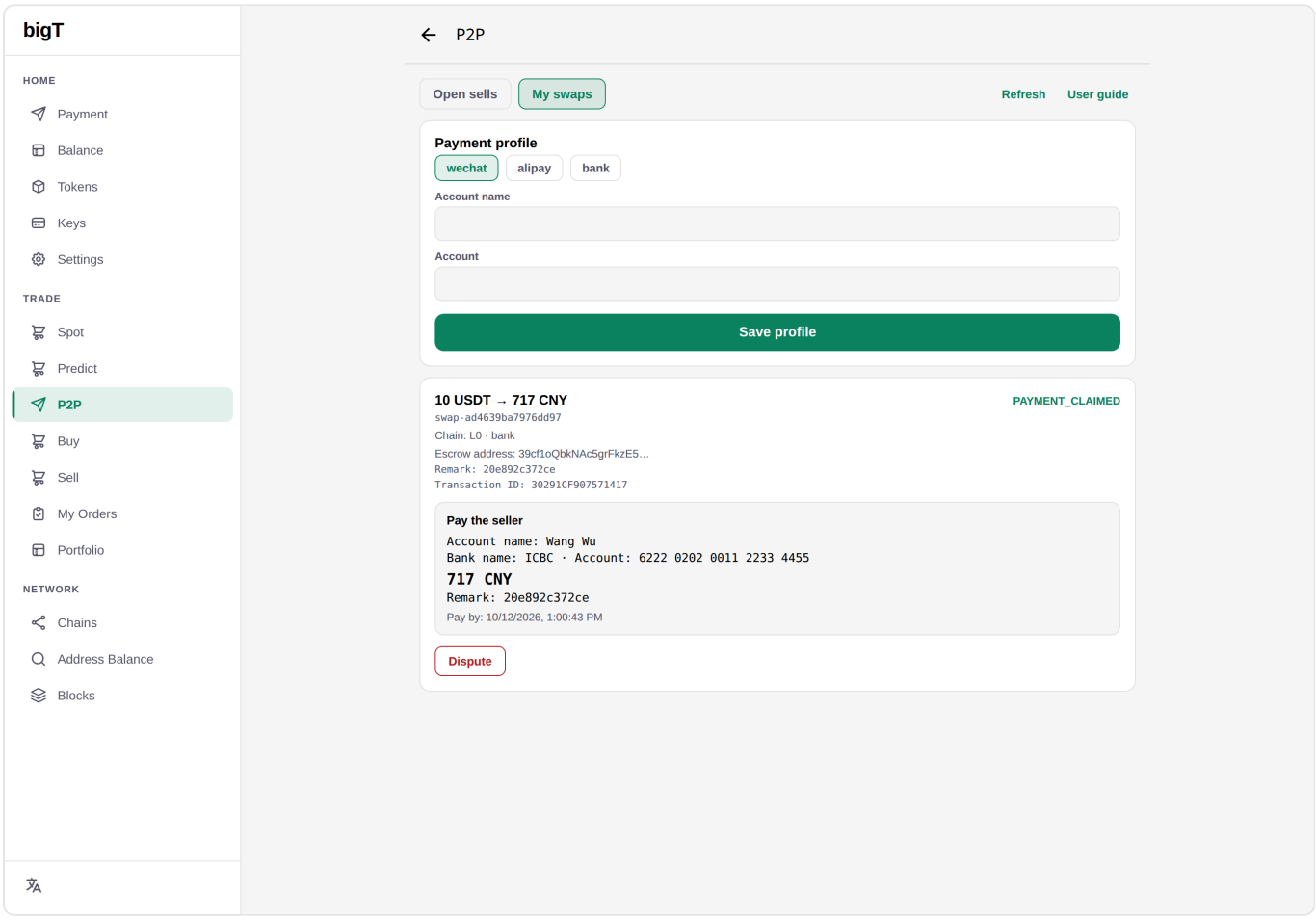





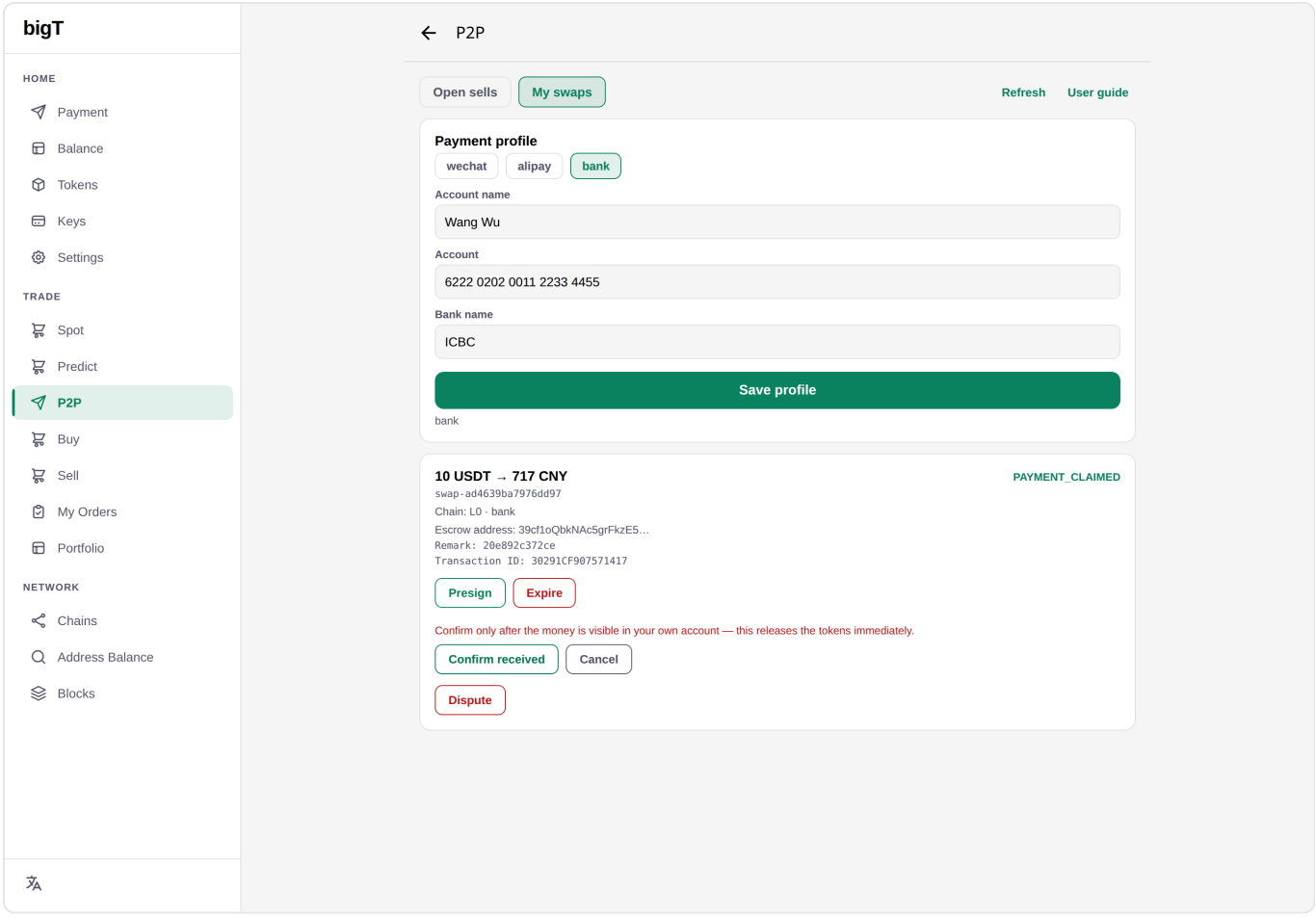
Step 6 — Buyer transfers 717 CNY and claims with the bank reference

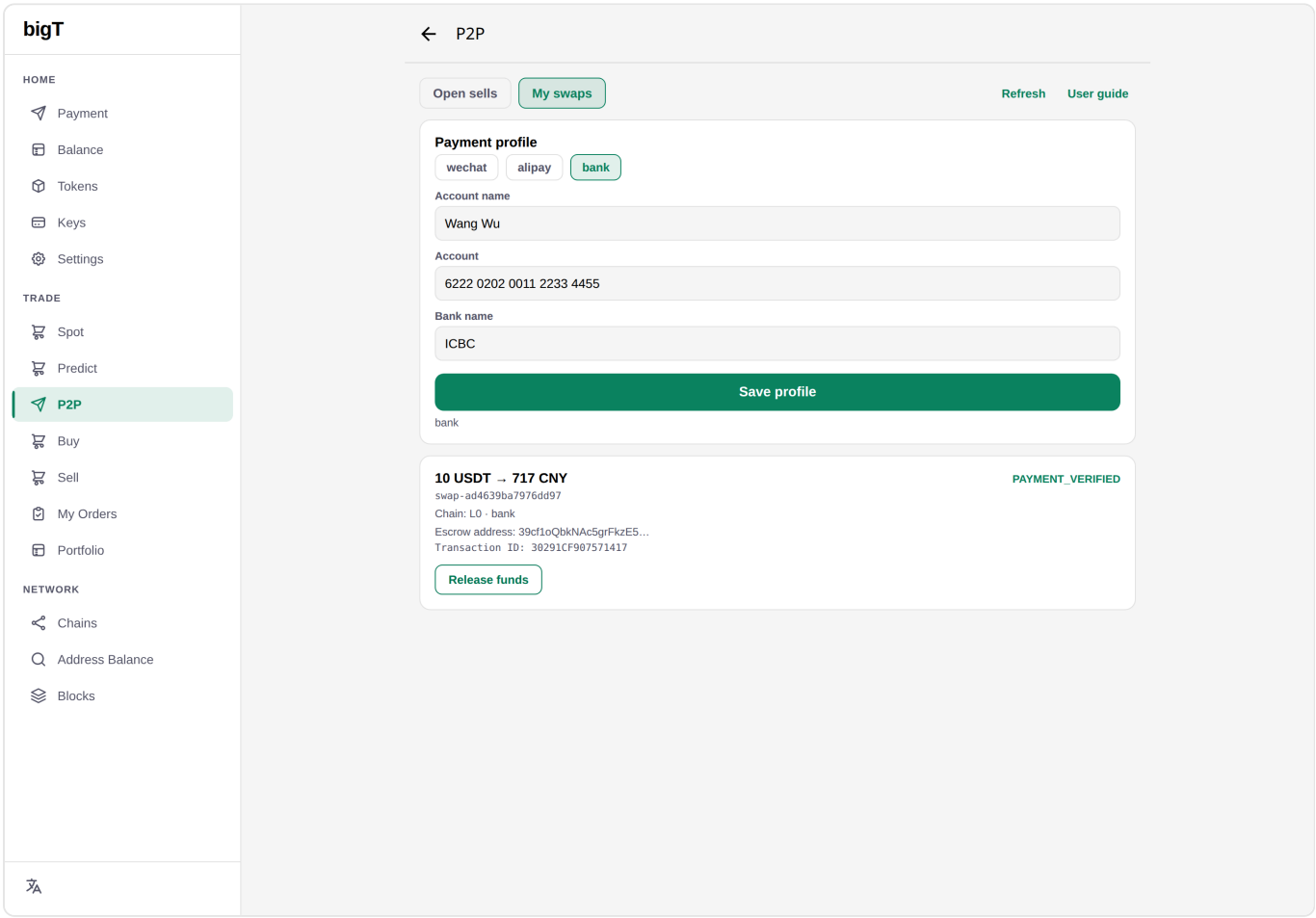
The buyer wires `717 CNY` from their own online banking, pastes the transfer reference (流水号, `30291CF907571417` in this run), and presses **I have paid**.


bigT
HOME
Payment
Balance
Tokens
Keys
Settings
TRADE
Spot
Predict
P2P
Buy
Sell
My Orders
Portfolio
NETWORK
Chains
Address Balance
Blocks
P2P
Open sells
My swaps
Refresh
User guide
Payment profile
wechat
alipay
bank
Account name
Account
Save profile
10 USDT → 717 CNY
PAYMENT_PENDING
swap-ad4639ba7976dd97
Chain: L0 · bank
Escrow address: 39cf1oQbkNAc5grFkzE5…
Remark: 20e892c372ce
Pay the seller
Account name: Wang Wu
Bank name: ICBC · Account: 6222 0202 0011 2233 4455
717 CNY
Remark: 20e892c372ce
Pay by: 10/12/2026, 1:00:43 PM
Transaction ID
30291CF907571417
I have paid
Dispute

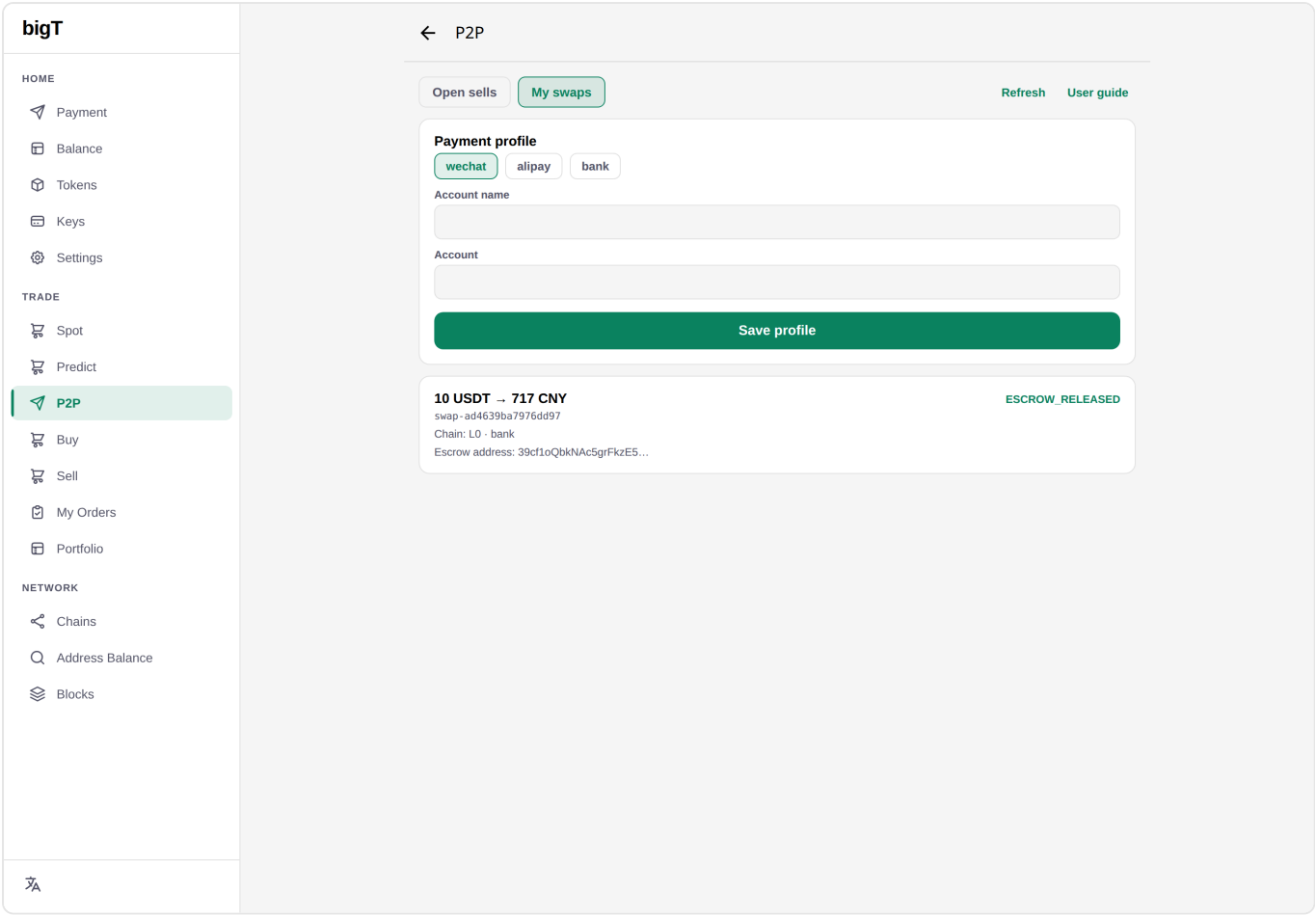

bigT
HOME
Payment
Balance
Tokens
Keys
Settings
TRADE
Spot
Predict
P2P
Buy
Sell
My Orders
Portfolio
NETWORK
Chains
Address Balance
Blocks
P2P
Open sells
My swaps
Refresh
User guide
Payment profile
wechat
alipay
bank
Account name
Account
Save profile
10 USDT → 717 CNY
PAYMENT_CLAIMED
swap-ad4639ba7976dd97
Chain: L0 · bank
Escrow address: 39cf1oQbkNAc5grFkzE5…
Remark: 20e892c372ce
Transaction ID: 30291CF907571417
Pay the seller
Account name: Wang Wu
Bank name: ICBC · Account: 6222 0202 0011 2233 4455
717 CNY
Remark: 20e892c372ce
Pay by: 10/12/2026, 1:00:43 PM
Dispute

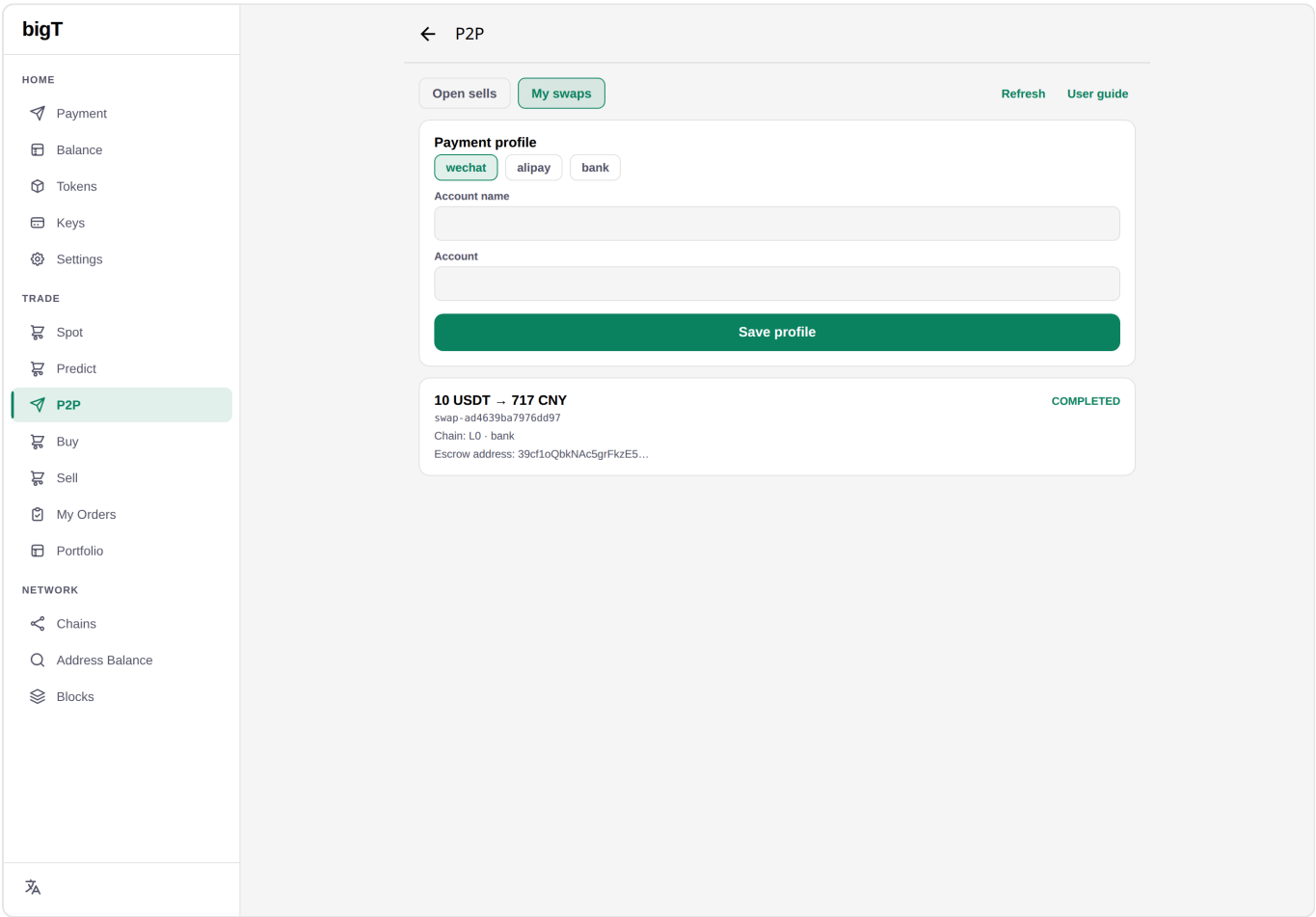
Step 7 — Seller checks their bank statement, then confirms





Step 8 — Release + complete (engine-signed)





7. Timeline of the captured runs

Time	Step
10:40:24	**wechat:** both wallets created
10:40:28	seller payment profile saved (`wxid_pay8888`)
10:40:32	sell order listed (`wantRail=wechat` , 715 CNY)
10:40:35	buyer matched the order (receive address only)
10:41:45	paced 70 s past the rate-limit window
10:41:47	escrow lock recorded (`ESCROW_LOCKED`)
10:41:50	buyer pulled payment instructions (户名 / account / 715 CNY / remark)
10:41:51	buyer claimed payment (流水号 `42034516068EAEC2`)
10:41:54	seller confirmed receipt (`PAYMENT_VERIFIED`) → release → complete
10:42:19	**alipay:** both wallets created
10:42:24	seller payment profile saved (`13800138000`)
10:42:27	sell order listed (`wantRail=alipay` , 716 CNY)
10:43:44	escrow lock recorded (`ESCROW_LOCKED`)
10:43:48	buyer claimed payment (trade no. `200E31282969161F6B32A2CF1C93`)
10:43:52	swap completed (`COMPLETED`)
10:44:17	**bank:** both wallets created
10:44:22	seller payment profile saved (`6222 0202 0011 2233 4455` , ICBC)
10:44:26	sell order listed (`wantRail=bank` , 717 CNY)
10:45:41	escrow lock recorded (`ESCROW_LOCKED`)
10:45:46	buyer claimed payment (bank ref. `30291CF907571417`)
10:45:50	swap completed (`COMPLETED`) — seller already holds the 717 CNY

(The 70 s gap after matching on each rail is deliberate: the engine allows 10 signed requests per DID per 60 s and the seller is at that ceiling after profile + list + match. Full timestamps are in `e2e/demo-output/p2p-cny-capture.json` .)

8. Device-side verification — the payee's own phone

A CNY transfer has no webhook, so the seller's own account is the receipt. The wallet/engine can **read it automatically** through the on-phone agent (`android-remote-control-mcp`),

so the seller does not have to eyeball the statement by hand. The user pairs the phone once (endpoint + capability token, revocable); the pairing **is** the consent.

Two reads satisfy the same `DeviceGateway`:

- **Screen read (accessibility).** Alipay and bank apps expose their UI tree; the worker reads the entry as text. WeChat **hides its window from accessibility**, so for WeChat use the notification read.
- **Receipt-notification read.** The payee's phone receives WeChat Pay's own “收款到账” push; the worker reads the notification shade and parses amount / 备注 / 交易单号 / time from it. This works even though WeChat hides its UI.



*Device capture. The notification text is illustrative — no real transfer was made for this shot; on a live swap it carries the swap's own `remark` and

交易单号.*

The engine drives it (docs/p2p.md §12):

```
POST /pairings                       → register the endpoint (DID-signed grant)
POST /swaps/:id/device-read {via}    → engine reads the caller's own side
POST /swaps/:id/verify               → the wallet signs the verdict
```

`device-read` takes `via: "screen" | "notification"` and returns a verdict (`observed: sent|received|none`) checked against the swap (exact amount, the per-swap `remark`, the time window, the 交易号). Under `SETTLEMENT_CNY_AUTO_RELEASE_ON` the engine may then act **without** the manual *Confirm* tap:

Policy	Machine confirmation
`off` (default)	verdicts are dispute evidence only
`seller_receipt`	the **seller's** own `received` read stands in for the confirm tap
`two_sided`	the buyer's `sent` **and** the seller's `received` must agree on one 交易号
`verified_receipt`	a bank 电子回单 that passed the bank's public 回单 verification settles it alone

A **notification** seller read qualifies as `seller_receipt`. Trust is tiered: one-sided evidence never machine-confirms under `two_sided`, and only the **payee's own** device is the side that counts. This is locked by `services/p2p-engine/test/wechat-mcp-verify.e2e.test.ts` — a mock `android_*` MCP endpoint serves the WeChat screen **and** the 收款到账 push, and the engine runs the full read → verdict → `/verify` → machine-confirm to `PAYMENT_VERIFIED`.

---

9. HTTP API reference (CNY rails)

All mutating endpoints take a signed body:

```
{ /* endpoint-specific fields, e.g. action / txId / … */
  "did": "did:key:z446G…",         // ML-DSA-87 did:key — the signer (see table §1)
  "nonce": "b1f0…",                // ≥16 chars, single use
  "timestamp": 1791509100000,      // ms, ±5 min (sign.ts NONCE_MAX_SKEW_MS)
  "signature": "…hex SignatureBundle…" }
```

`signature` = **PQ**: `SHA256(JSON.stringify(payload, sortedKeys))` signed with the did's ML-DSA-87 key, hex-serialized `SignatureBundle` (`sign.ts:39-61`, mirrors the wallet's `p2pIdentity.signP2pPayload`). A classic did instead verifies an Ed25519 signature over the raw bytes, or an ECDSA signature over their sha256 — supported for compatibility, not what new keys use. `payload` is always the body **without** `signature`.

Endpoint	Signer	Required fields	Success	
`GET /healthz`	—	—	`{ ok: true }`	
`GET /public/orders`	—	—	`{ orders: […] }` (no signature, no PII)	
`GET /capabilities`	—	—	`{ rails: […] }` (which rails this engine serves)	
`POST /profiles`	seller	`method`, `accountName`, `account` (+ `bankName`, `qr`)	`{ ok, method, updatedAt }`	
`POST /profiles/mine`	seller	—	`{ profiles: […] }`	
`POST /orders`	seller	`giveToken`, `giveAmount`, `giveChain`, `wantCurrency`, `wantAmount`, `wantRail`, `validUntil`	`201 { orderId, status }`	
`POST /orders/:id/match`	buyer	`receiveAddress`	`201 { swapId, status, escrowAddress }`	
`POST /swaps/mine`	party	—	`{ swaps: […] }`	
`POST /swaps/:id/transitions`	seller (lock/expire/refund) · either (cancel) · **engine** (release/complete)	`action`, + `txHash` for lock/release	`{ swapId, status }`	
`POST /swaps/:id/payment-instructions`	buyer	—	`201 { … instructions, remark, payBy }`	
`POST /swaps/:id/proof`	buyer	`txId` (1..64 `[A-Za-z0-9._-]`), optional `remark`, `receipt`, `receiptSha256`	`{ swapId, status, txId, receiptSha256 }`	
`POST /swaps/:id/confirm`	seller	—	`{ swapId, status: "PAYMENT_VERIFIED" }`	
`POST /swaps/:id/dispute`	either party	optional `reason` (≤512)	`{ swapId, dispute }`	
`POST /swaps/:id/dispute/resolve`	admin token	`outcome=release\`	refund`	`{ swapId, … }`

Guard rails the engine enforces on every one of them:

- `409` when the status transition is illegal (`state.ts` `VALID_TRANSITIONS`),

`409` on a dedicated action sent to the generic route (`instructions` ,

`payment_proof` , `payment_confirm` are rejected there),

- `403` unless `did` is the required signer, `400` on bad signature, replayed

nonce, stale timestamp, or `rate limit exceeded` (10/min/DID),

- `422 swap frozen: …` while a dispute is open for the forward actions,
- `400` for rail/action mismatches.

---

10. Review findings (code review of the CNY implementation)

Verified good: the transition table keeps `PAYMENT_CLAIMED` out of the release path; `release` / `complete` are engine-only; the profile is party-scoped; the remark is regenerated per swap and checked on `proof` ; the receipt hash is recomputed server-side; unit tests cover the CNY legs (`services/p2p-engine/test/cny.test.ts`).

Open gaps, in the order they matter:

1. **No automatic release after `confirm` .** The engine does not issue `release` / `complete` itself when the seller confirms — an operator/cron must do it with the engine key. Today the wallet UI has no button for it either (by design: the wallet must not hold the engine key).
2. **`payBy` is advisory.** The `SETTLEMENT_CNY_REMARK_TTL` deadline is returned to the buyer but never enforced — a swap can be claimed after it expires, and `SETTLEMENT_CNY_MAX_CENTS` / a confirm timeout are not implemented.
3. **`receiptSha256` hashed the data-URL string, not the image bytes** — fixed: `server.ts` decodes the `…` payload and hashes the decoded bytes (400 on malformed base64), so the digest is reproducible with `sha256 <file>` ; `test/cny.test.ts` asserts it.
4. **`Alert.alert` is a no-op on react-native-web** — every error/success message in the P2P screen (including the payment instructions) was invisible in the web build. Fixed: inline notice banner + inline instructions panel + confirm warning, localized in 12 languages (`p2p.payBy` , `p2p.confirmWarn`).
5. **Rate limit bites scripted flows.** 10 signed calls/DID/minute is tight for the wallet's own read-modify loops (`/swaps/mine` after every transition); the capture script has to pace itself. Worth making `RATE_LIMIT` an env knob if real users ever hit it.

---

11. Regenerating this guide

```
( cd e2e && RAILS=wechat,alipay,bank node capture-p2p-cny.mjs )  # CNY legs (14 shots per rail)
node docs/p2p-demo/scripts/gen-p2p-all-pdf.mjs
# → docs/p2p-demo/assets/p2p-all.pdf
```

Or `./p2pdemo.sh capture walkthrough`. The generator is dependency-free (own

markdown → HTML, images embedded as data URIs, `chromium --headless

--print-to-pdf`), the same pipeline the dai help guides use.